МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»
Пермский институт (филиал)

Отчет
по производственной практике

ПП.02.01 Технология разработки программного обеспечения
(индекс по РУП и наименование практики)

Профессионального модуля ПМ.02 Осуществление интеграции программных модулей
(индекс по РУП и наименование профессионального модуля)

Специальность 09.02.07 Информационные системы и программирование

Обучающийся _____ Паршакова Наталья Алексеевна
(подпись) (фамилия, имя, отчество)

Группа ИПс-31

Руководитель практической подготовки от профильной организации

Индивидуальный предприниматель
(должность)

_____ Паршакова Екатерина Николаевна
(подпись) (фамилия, имя, отчество)

БП «16» марта 2026 года

Руководитель практической подготовки от института

_____ Берестов Дмитрий Борисович
(подпись) (фамилия, имя, отчество)

«16» марта 2026 год

Пермь, 2026 год

Содержание

ВВЕДЕНИЕ 4

РАЗДЕЛ 1. ОРГАНИЗАЦИОННО-ПОДГОТОВИТЕЛЬНЫЙ ЭТАП ПРОХОЖДЕНИЯ ПРАКТИКИ НА ПРЕДПРИЯТИИ 6

Тема 1.1. Инструктаж по прохождению производственной практики и правилам безопасности работы на предприятии 6

Раздел 2. Ознакомление со структурой и характером деятельности подразделения 8

Тема 2.1 Ознакомление с организацией работы на предприятии или в структурном подразделении 8

Тема 2.2 Ознакомление с должностными и функциональными обязанностями 9

РАЗДЕЛ 3. РАБОТА НА РАБОЧИХ МЕСТАХ ИЛИ В ПОДРАЗДЕЛЕНИЯХ ПРЕДПРИЯТИЯ 12

Тема 3.1. Выполнение производственных заданий 12

3.1.1. Использование выбранной системы контроля версий 12

3.1.2. Разработка программного обеспечения 13

3.1.2.1 Выбор модели разработки 13

3.1.2.2 Инструменты, использованные для создания продукта 14

3.1.2.3 Среда программирования 14

3.1.2.4 Этапы разработки программного обеспечения 15

3.1.3. Использование методов для получения кода с заданной функциональностью и степенью качества 51

3.1.4. Инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования 53

3.1.5. Выполнение отладки программного модуля с использованием специализированных программных средств; 54

3.1.6. Осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения 55

3.1.7. Интегрирование программных модулей 59

3.1.8. Верификация и аттестация программного обеспечения 60

ЗАКЛЮЧЕНИЕ 62
СПИСОК ЛИТЕРАТУРЫ..... 64

ВВЕДЕНИЕ

Практика имеет целью комплексное освоение студентами всех видов профессиональной деятельности по специальности 09.02.07 «Информационные системы и программирование», формирование общих и профессиональных компетенций, а также приобретение необходимых умений и опыта практической работы по специальности.

Производственная практика направлена на освоение обучающимися профессиональных компетенций в процессе самостоятельного выполнения определенных видов работ, необходимых для осуществления профессиональной деятельности в максимально приближенных к производству условиях, в рамках модулей ППССЗ по основным видам профессиональной деятельности для освоения методов и приемов практического применения прикладных программных продуктов для программного обеспечения компьютерных систем.

В настоящей работе представлен отчет о прохождении производственной практики студента-практиканта. Основными видами деятельности которой являлись: разработка требования к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент; моделирование процесса разработки программного обеспечения

Задачи практики (освоение обучающимися профессиональных и общих компетенций):

1. Выполнять интеграцию модулей в программное обеспечение;
2. Выполнять отладку программного модуля с использованием специализированных программных средств;
3. Осуществлять разработку тестовых наборов и тестовых сценариев для программного обеспечения;
4. Производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования;

5. Выбирать способы решения задач профессиональной деятельности, применительно к различным контекстам;

6. Осуществлять поиск, анализ и интерпретацию информации, необходимой для выполнения задач профессиональной деятельности

7. Планировать и реализовывать собственное профессиональное и личностное развитие;

8. Работать в коллективе и команде, эффективно взаимодействовать с коллегами, руководством, клиентами

9. Осуществлять устную и письменную коммуникацию на государственном языке Российской Федерации с учетом особенностей социального и культурного контекста.

10. Использовать информационные технологии в профессиональной деятельности

11. Пользоваться профессиональной документацией на государственном и иностранном языках

РАЗДЕЛ 1. ОРГАНИЗАЦИОННО-ПОДГОТОВИТЕЛЬНЫЙ ЭТАП ПРОХОЖДЕНИЯ ПРАКТИКИ НА ПРЕДПРИЯТИИ

Тема 1.1. Инструктаж по прохождению производственной практики и правилам безопасности работы на предприятии

В первый день прохождения производственной практики на базе Индивидуального предпринимателя Паршаковой Екатерины Николаевны (ИП Паршакова Е.Н.) руководителем от организации был проведен вводный и первичный инструктаж по охране труда и технике безопасности.

В ходе инструктажа я была ознакомлена с общими функциональными обязанностями практиканта, правилами поведения на рабочем месте, а также требованиями безопасности при работе с электрооборудованием. Особое внимание было уделено правилам техники безопасности при работе с персональным компьютером и периферийными устройствами, правилам противопожарной безопасности, действиям в чрезвычайных ситуациях и безопасным методам работы с электрическими приборами во избежание поражения электрическим током.

Руководителем практики были разъяснены основные положения инструкции по охране труда для пользователей ПЭВМ, включая требования к организации рабочего места, режиму труда и отдыха, а также правила электробезопасности при эксплуатации компьютерной техники.

Описание рабочего места и технического обеспечения

Для выполнения заданий по интеграции программных модулей и разработке приложения для учёта партнёров и их продаж рабочее место практиканта было оснащено современным персональным компьютером с операционной системой Windows 10, обеспечивающей совместимость с необходимым инструментарием разработки.

В качестве системы контроля версий использовался Git, который применялся для управления исходным кодом проекта, отслеживания изменений и обеспечения целостности истории разработки программных модулей. Системой управления базами данных выбрана PostgreSQL, которая использовалась для создания базы данных и проектирования её структуры.

Интегрированной средой разработки стала Microsoft Visual Studio 2022, в которой создавалось комплексное решение, включающее библиотеку классов для реализации бизнес-логики, WPF-приложение для пользовательского интерфейса и тестовый проект для модульного тестирования. Для доступа к данным применялась технология Entity Framework Core, обеспечивающая объектно-реляционное отображение при интеграции с базой данных.

Таким образом, рабочее место было полностью подготовлено для выполнения задач по интеграции программных модулей, разработке приложения для учёта партнёров и продаж, его отладке и тестированию в условиях, максимально приближенных к реальной профессиональной деятельности.

Раздел 2. Ознакомление со структурой и характером деятельности подразделения

Тема 2.1 Ознакомление с организацией работы на предприятии или в структурном подразделении

В рамках данной темы мною было проведено ознакомление с организацией работы на предприятии ИП Паршакова Е.Н., специализирующемся на консультационных услугах и работах в области компьютерных технологий.

Изучение режима работы и формы организации труда

На предприятии установлен нормированный рабочий день: пятидневная рабочая неделя с понедельника по пятницу с 9:00 до 18:00, с перерывом на обед с 13:00 до 14:00. Формой организации труда является индивидуальное выполнение проектных задач с регулярными совещаниями для обсуждения прогресса разработки и вопросов интеграции программных модулей.

Ознакомление с правилами внутреннего распорядка

Правила внутреннего трудового распорядка предприятия включают соблюдение установленного режима рабочего времени, своевременное и качественное выполнение распоряжений руководителя, бережное отношение к имуществу работодателя, соблюдение трудовой дисциплины, а также обеспечение сохранности конфиденциальной информации, включая исходные коды разрабатываемого программного обеспечения.

Изучение структуры предприятия и штатного расписания

Предприятие ИП Паршакова Е.Н. имеет упрощенную организационную структуру, характерную для малого бизнеса в сфере ИТ-консалтинга. Руководство всей деятельности определяет непосредственно индивидуальный

предприниматель. Четко выделенного ИТ-подразделения как отдельной структурной единицы нет, однако все сотрудники и привлеченные специалисты задействованы в выполнении проектных задач по разработке и интеграции программных модулей для автоматизации учётных задач клиентов.

Принципы управления и осуществления должностных обязанностей

Управление деятельностью осуществляется на принципах единоначалия. Индивидуальный предприниматель определяет стратегию развития, распределяет задачи между исполнителями и контролирует их выполнение. Взаимодействие с заказчиком строится на основе проектного подхода: выявляются потребности клиента в автоматизации, формируются требования к программным модулям, ставятся задачи перед разработчиками, которые несут ответственность за их техническую реализацию, интеграцию модулей и качество конечного продукта.

Тема 2.2 Ознакомление с должностными и функциональными обязанностями

В рамках данной темы мною была изучена должностная инструкция программиста-разработчика, функционал которого наиболее близок к задачам, выполняемым в ходе практики по интеграции программных модулей.

Изучение должностной инструкции сотрудника

Согласно инструкции, программист-разработчик относится к категории специалистов, назначается на должность и освобождается от нее приказом индивидуального предпринимателя и подчиняется непосредственно руководителю.

Описание прав и обязанностей специалиста

Программист-разработчик выполняет следующие должностные обязанности: проводит анализ бизнес-процессов клиента и разрабатывает требования к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент; осуществляет интеграцию программных модулей в единое программное обеспечение, обеспечивая их корректное взаимодействие; выполняет отладку программных модулей с использованием специализированных программных средств; разрабатывает тестовые наборы и тестовые сценарии для проверки работоспособности программного обеспечения; производит инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования; осуществляет диагностику проблем в работе информационных систем клиентов и предлагает пути их устранения; подготавливает отчётную и техническую документацию по выполненным работам.

Программист-разработчик имеет право получать информацию, необходимую для решения поставленных задач по разработке и интеграции модулей, использовать системы контроля версий для управления исходным кодом, вносить предложения по совершенствованию архитектуры программных модулей и процессов их интеграции, знакомиться с проектами решений руководства, касающимися его деятельности, а также требовать от руководства создания нормальных условий для выполнения служебных обязанностей.

Согласование с руководителем практики индивидуального задания

Совместно с руководителем практики от предприятия было сформировано и согласовано индивидуальное задание, направленное на приобретение практического опыта в области разработки и интеграции программных модулей. Задание предусматривало разработку программного модуля для автоматизации учёта партнёров и их продаж, включая проектирование базы данных,

реализацию бизнес-логики, создание пользовательского интерфейса, интеграцию компонентов, отладку и тестирование.

Формулировка целей и задач практики

На основе индивидуального задания были сформулированы следующие цели и задачи практики. Целью практики является комплексное освоение видов профессиональной деятельности по интеграции программных модулей и приобретение практического опыта разработки, отладки и тестирования программного обеспечения для учёта партнёров и их продаж.

Задачами практики выступают проведение анализа предметной области и разработка требований к программным модулям для учёта партнёров и продаж на основе анализа проектной документации, проектирование логической модели данных и создание базы данных в PostgreSQL, отражающей основные бизнес-сущности, разработка программных модулей, включающих библиотеку классов для бизнес-логики и WPF-приложение для пользовательского интерфейса, выполнение интеграции разработанных модулей в единый программный комплекс, проведение отладки программного обеспечения с использованием специализированных средств среды разработки, разработка тестовых наборов и сценариев, выполнение модульного тестирования для проверки корректности работы основных функций, а также проведение инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования.

РАЗДЕЛ 3. РАБОТА НА РАБОЧИХ МЕСТАХ ИЛИ В ПОДРАЗДЕЛЕНИЯХ ПРЕДПРИЯТИЯ

Тема 3.1. Выполнение производственных заданий

3.1.1. Использование выбранной системы контроля версий

В качестве распределенной системы контроля версий (РСКВ) для управления исходным кодом проекта выбрана система **Git**. Данный выбор обусловлен ее архитектурными преимуществами: полная распределенность, криптографическая целостность данных и эффективная модель ветвления.

Архитектура и принципы функционирования

Git относится к классу **распределенных систем контроля версий**, что принципиально отличает ее от централизованных аналогов (Subversion, CVS). Каждый локальный репозиторий содержит полную копию истории разработки, что обеспечивает:

- автономность работы разработчика;
- отказоустойчивость хранения данных;
- децентрализованную модель совместной разработки.

Концептуальной основой Git является **модель потоков снимков (snapshots)**, а не хранение изменений (дельт). При каждом коммите система фиксирует текущее состояние всех отслеживаемых файлов, создавая целостный снимок проекта.

Объектная модель данных

Целостность данных в Git обеспечивается криптографическим хешированием (алгоритм SHA-1). Все объекты репозитория идентифицируются контрольными суммами, что гарантирует неизменность истории при попытках фальсификации. Основные типы объектов:

- blob — содержимое файла;
- tree — структура директории;
- commit — снимок состояния с метаданными;

– tag — именованная ссылка на коммит.

Модель ветвления

Ветки в Git реализованы как легковесные указатели на коммиты, что позволяет создавать и переключать ветки без значительных накладных расходов. Это обеспечивает гибкость параллельной разработки и изоляцию экспериментальных изменений.

3.1.2. Разработка программного обеспечения

3.1.2.1 Выбор модели разработки

При разработке программного обеспечения для учёта партнёров и их продаж была выбрана итерационно-инкрементальная модель разработки. Данный подход предполагает поэтапное создание продукта небольшими инкрементами, каждый из которых добавляет новую функциональность к уже существующей. Это позволяет последовательно уточнять требования, оперативно реагировать на замечания и постепенно повышать качество решения.

На ранних итерациях формируется минимально работоспособная версия системы, реализующая базовые функции работы с партнёрами и их продажами. На последующих этапах выполняется расширение функционала, доработка интерфейса, внедрение дополнительных проверок и тестов. Итерационно-инкрементальная модель хорошо подходит для учебных проектов, так как обеспечивает наглядность процесса разработки и позволяет своевременно выявлять ошибки.

В рамках выбранной модели каждая итерация включает в себя анализ и уточнение требований, проектирование отдельных компонент, реализацию и модульное тестирование, а также интеграцию в существующую систему. Такой цикл повторяется до достижения требуемого уровня функциональной полноты и качества программного обеспечения. Преимуществом данного подхода является снижение рисков: даже при ограниченных сроках разработчик

получает работоспособный продукт, реализующий ключевые задачи. Кроме того, постепенное наращивание функциональности облегчает понимание архитектуры и упрощает сопровождение программного обеспечения.

3.1.2.2 Инструменты, использованные для создания продукта

Для реализации программного продукта был использован набор инструментов, обеспечивающих полный цикл разработки: от проектирования базы данных до создания пользовательского интерфейса и модульного тестирования.

Для работы с сервером баз данных применялась система управления базами данных PostgreSQL, предоставляющая развитые средства для создания таблиц, определения связей, выполнения запросов и администрирования. Управление структурой базы данных и выполнение SQL-команд осуществлялись через консольные утилиты и графические средства, входящие в состав СУБД.

Для реализации серверной части и бизнес-логики применялась платформа .NET с использованием языка программирования C#. В качестве основной интегрированной среды разработки была выбрана Visual Studio, обеспечивающая поддержку работы с решениями, проектами, менеджером пакетов, отладчиком и средствами тестирования.

Для доступа к данным использовалась технология объектно-реляционного отображения, что позволило работать с записями таблиц как с объектами в коде, не опираясь непосредственно на текст SQL-запросов. Дополнительно применялись инструменты управления зависимостями и внешними библиотеками, а также средства контроля версий (при необходимости), что соответствует современным требованиям к организации процесса разработки.

3.1.2.3 Среда программирования

В качестве основной среды программирования была использована интегрированная среда разработки Visual Studio, ориентированная на создание

приложений под платформу .NET. Данная среда предоставляет широкий набор инструментов: редактор исходного кода с подсветкой синтаксиса и автодополнением, средства отладки, визуальные конструкторы пользовательских интерфейсов, а также встроенную поддержку модульного тестирования. Наличие мощного отладчика позволяет оперативно выявлять и устранять ошибки, анализировать состояние переменных и последовательность выполнения программы.

Для разработки графического пользовательского интерфейса использовалась технология WPF, поддерживаемая Visual Studio. Она предоставляет возможности визуального проектирования окон, настройки привязок данных, стилизации элементов интерфейса и разделения представления и логики. Работа с проектами библиотеки классов, тестовыми проектами и клиентским приложением велась в рамках единого решения, что упростило навигацию между компонентами и обеспечило удобную структуру проекта. Такой выбор среды программирования позволил реализовать функциональное и наглядное приложение в сжатые сроки и с соблюдением требований к качеству.

3.1.2.4 Этапы разработки программного обеспечения

Настройка и создание БД

Разработка программного обеспечения для работы с партнёрами начинается с проектирования и создания базы данных, обеспечивающей надёжное хранение и обработку информации о контрагентах, их продажах и скидках. В рамках проекта была использована система управления базами данных PostgreSQL, так как она обеспечивает высокую производительность, поддержку транзакций и развитые средства работы с типами данных.

Прежде чем приступить к созданию структуры базы данных, необходимо подготовить инфраструктуру для хранения данных. На первом этапе в PostgreSQL создаётся отдельное табличное пространство, в котором будут

размещаться файлы данных проекта. Для этого в консоли PostgreSQL выполняется команда:

CREATE TABLESPACE parshakova_tb OWNER app LOCATION 'C:/TB_DB';

В данной команде ключевое слово CREATE TABLESPACE создаёт новое табличное пространство с именем parshakova_tb. Параметр OWNER app указывает, что владельцем табличного пространства является пользователь app, а параметр LOCATION определяет путь к каталогу на диске, где будут физически храниться файлы базы данных. Это позволяет логически отделить данные проекта от других баз и при необходимости упростить их резервное копирование и перенос. (См. Рис 1)

```
postgres=# CREATE TABLESPACE parshakova_tb OWNER app LOCATION 'C:/TB_DB';
CREATE TABLESPACE
```

Рисунок 1 – Создание табличного пространства

На следующем этапе создаётся отдельная база данных, которая будет использовать подготовленное табличное пространство. Для этого выполняется команда:

CREATE DATABASE parshakova_db WITH OWNER app TABLESPACE parshakova_tb;

В данной команде указывается имя создаваемой базы данных parshakova_db, владелец app и ранее созданное табличное пространство parshakova_tb. В результате все объекты данной базы данных будут размещаться в указанном табличном пространстве. (См. Рис 2)

```
postgres=# CREATE DATABASE parshakova_db WITH OWNER app TABLESPACE parshakova_tb;
CREATE DATABASE
```

Рисунок 2 – Создание базы данных

После создания базы данных необходимо перейти к работе именно с ней. Подключение к базе данных выполняется с помощью служебной команды psql:

\c parshakova_db

Далее создаётся отдельная схема app, в которой будут размещаться все таблицы и другие объекты приложения. Это позволяет логически отделить прикладные объекты от служебных и системных. Создание и назначение владельца выполняются следующими командами:

CREATE SCHEMA app AUTHORIZATION app; ALTER SCHEMA app OWNER TO app;

В результате в базе данных parshakova_db создаётся схема app, права на которую принадлежат пользователю app. После этого производится повторное подключение к базе под пользователем app:

\c parshakova_db app

Таким образом, дальнейшее создание таблиц и работа с данными выполняются от имени прикладного пользователя, что соответствует требованиям безопасной организации доступа. (См. Рис 3)

```
postgres=# \c parshakova_db
Вы подключены к базе данных "parshakova_db" как пользователь "postgres".
parshakova_db=# CREATE SCHEMA app AUTHORIZATION app;
CREATE SCHEMA
parshakova_db=# ALTER SCHEMA app OWNER TO app;
ALTER SCHEMA
```

Рисунок 3 – Создание и настройка схемы app

Структура базы данных спроектирована с учётом предметной области «партнёры и их продажи». В рамках проекта используются три основные таблицы: partner, sale и discount. Таблица app.partner предназначена для хранения сведений о партнёрах: идентификатора, типа организации, наименования, директора, контактных данных, юридического адреса и рейтинга. Её создание выполняется следующей SQL-командой:

CREATE TABLE app.partner (

id serial4 PRIMARY KEY,

"type" text NOT NULL,

name text NOT NULL,

director text NOT NULL,

email text NOT NULL,

phone text NOT NULL,

legaladdress text NOT NULL,

rating int4 NULL);

Поле id является первичным ключом и автоматически заполняется с использованием последовательности. Поля type, name, director, email, phone и legaladdress объявлены как NOT NULL, что предотвращает появление неполных записей в таблице партнёров. Поле rating хранит целочисленную оценку, характеризующую важность или надёжность партнёра, и может быть пустым на начальном этапе. (См. Рис 4)

```
parshakova_db=> CREATE TABLE app.partner (
parshakova_db(>     id serial4 PRIMARY KEY,
parshakova_db(>     "type" text NOT NULL,
parshakova_db(>     name text NOT NULL,
parshakova_db(>     director text NOT NULL,
parshakova_db(>     email text NOT NULL,
parshakova_db(>     phone text NOT NULL,
parshakova_db(>     legaladdress text NOT NULL,
parshakova_db(>     rating int4 NULL
parshakova_db(> );
```

Рисунок 4 – Создание таблицы partner

Для хранения информации о скидках, предоставляемых партнёрам, используется таблица app.discount. В ней хранятся идентификатор записи, ссылка на партнёра и значение скидки в процентах:

CREATE TABLE app.discount (

id serial4 PRIMARY KEY,

partnerid int4 REFERENCES app.partner(id) ON DELETE SET NULL,

percentage float8 NULL);

Поле partnerid является внешним ключом и ссылается на первичный ключ таблицы partner. Опция ON DELETE SET NULL задаёт поведение при удалении партнёра: связанные записи скидок сохраняются, но ссылка на удалённого

партнёра обнуляется. Это позволяет сохранить историю скидок, даже если сам партнёр больше не используется в системе. В поле percentage хранится значение скидки в виде числа с плавающей точкой двойной точности. (См. Рис 5)

```
parshakova_db=> CREATE TABLE app.discount (
parshakova_db(>     id serial4 PRIMARY KEY,
parshakova_db(>     partnerid int4 REFERENCES app.partner(id) ON DELETE SET NULL,
parshakova_db(>     percentage float8 NULL
parshakova_db(> );
CREATE TABLE
```

Рисунок 5 – Создание таблицы discount

Для фиксации истории продаж используется таблица app.sale. В ней содержатся сведения об идентификаторе продажи, партнёре, количестве реализованного товара, дате операции и наименовании продукта:

CREATE TABLE app.sale (

id serial4 PRIMARY KEY,

partnerid int4 REFERENCES app.partner(id) ON DELETE SET NULL,

quantity int4 NOT NULL,

"date" date NOT NULL,

product_name varchar NOT NULL);

Поле partnerid ссылается на таблицу partner и связывает конкретную продажу с конкретным партнёром. Поля quantity и date объявлены как NOT NULL и обеспечивают обязательное указание количества и даты продажи. Поле product_name хранит наименование реализованного товара в текстовом виде. (См. Рис 6)

```
parshakova_db=> CREATE TABLE app.sale (
parshakova_db(>     id serial4 PRIMARY KEY,
parshakova_db(>     partnerid int4 REFERENCES app.partner(id) ON DELETE SET NULL,
parshakova_db(>     quantity int4 NOT NULL,
parshakova_db(>     "date" date NOT NULL,
parshakova_db(>     product_name varchar NOT NULL
parshakova_db(> );
CREATE TABLE
```

Рисунок 6 – Создание таблицы sale

После создания таблиц выполняется проверка структуры базы данных в графических средствах администрирования. На схеме базы данных отображаются таблицы partner, sale и discount, а также связи между ними по внешним ключам. Таблица partner является центральной: с ней связаны таблицы sale и discount по полю partnerid. Это соответствует логике предметной области, в которой каждый партнёр может иметь множество продаж и одну или несколько записей о скидках. (См. Рис 7)

Рисунок 7 – Схема базы данных

На заключительном этапе работы с сервером баз данных выполняется экспорт структуры и данных в SQL-скрипт. Для этого используется как утилита командной строки, так и специализированные графические средства (например, pgAdmin или DBSchema). Полученный дамп позволяет при необходимости развернуть базу данных на другом сервере или восстановить её после сбоев. (См. Рис 8)

```
C:\Program Files\PostgreSQL\17\bin>pg_dump -U app -d parshakova_db -f D:\Praktika\parshakova_db_dump.sql
```

Рисунок 8 – Создание дампа базы данных

Разработка приложения

После создания и настройки базы данных выполняется разработка прикладного программного обеспечения, обеспечивающего удобный доступ к данным и автоматизацию обработки информации о партнёрах. Архитектура решения включает библиотеку классов на языке C#, реализующую бизнес-логику, и клиентское WPF-приложение, предоставляющее графический

интерфейс пользователя. Для доступа к базе данных используется технология Entity Framework Core и провайдер Npgsql.

Вначале создаётся проект библиотеки классов на языке C# с названием ParshakovaLib, ориентированный на используемую версию платформы .NET. В библиотеку добавляются необходимые пакеты NuGet, в том числе:

– Microsoft.EntityFrameworkCore;

– Npgsql.EntityFrameworkCore.PostgreSQL;

– Microsoft.EntityFrameworkCore.Tools (для удобства работы с миграциями и scaffolding).(см.Рис 9)

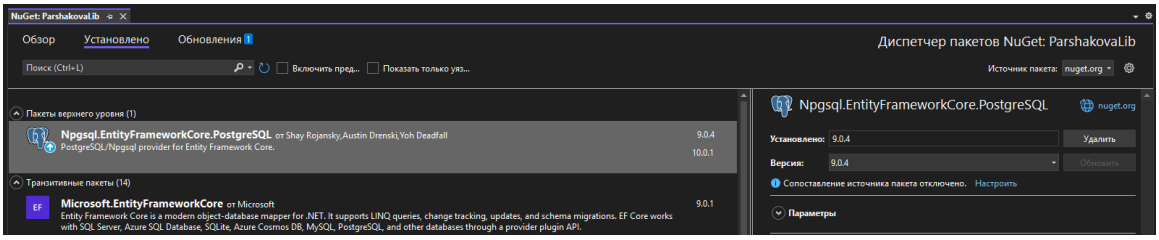

Рисунок 9 – Подключение пакетов NuGet

Далее выполняется генерация контекста данных и классов-сущностей на основе уже созданной базы данных parshakova_db. Для этого в консоли диспетчера пакетов Visual Studio выполняется команда вида:

Scaffold-DbContext "Host=localhost; Database=parshakova_db; Username=app; Password=123456789" Npgsql.EntityFrameworkCore.PostgreSQL

В результате в проекте ParshakovaLib автоматически создаются классы контекста ApplicationContext и сущностей Partner, Sale и Discount, соответствующие структуре таблиц базы данных. Кроме того, генерируется конфигурация сопоставления свойств классов с полями таблиц, а также связи между сущностями. (См. Рис 10)

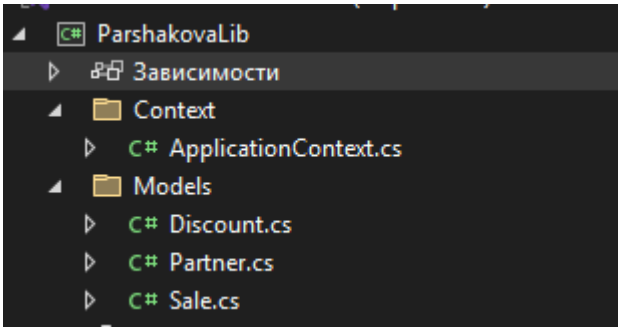

Рисунок 10 – Сгенерированные классы контекста и моделей

Для структурирования кода в библиотеке создаются пространства имён ParshakovaLib.Context и ParshakovaLib.Models. В папке Context располагается класс ApplicationContext, наследующий DbContext и отвечающий за подключение к базе данных. В папке Models размещаются классы Partner, Sale и Discount, описывающие сущности предметной области. В классе ApplicationContext задаются DbSet-свойства, соответствующие таблицам базы данных, а также конфигурируется строка подключения к PostgreSQL. (См. Рис 11)

```
using System;
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata;
using ParshakovaLib.Models;

namespace ParshakovaLib.Context
{

    public partial class ApplicationContext : DbContext
    {

        public ApplicationContext()
        {
        }


        public ApplicationContext(DbContextOptions<ApplicationContext> options)
            : base(options)
        {
        }


        public virtual DbSet<Discount> Discount { get; set; }
        public virtual DbSet<Partner> Partner { get; set; }
        public virtual DbSet<Sale> Sale { get; set; }

        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            if (!optionsBuilder.IsConfigured)
            {
                optionsBuilder.UseNpgsql("Host=localhost;Database=parshakova_db;Username=app;Password=123456789");
            }
        }

        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            modelBuilder.Entity<Discount>(entity =>
            {
                entity.ToTable("discount", "app");

                entity.Property(e => e.Id)
                    .HasColumnName("id")
                    .HasDefaultValueSql("nextval('discount_id_seq'::regclass)");

                entity.Property(e => e.Partnerid).HasColumnName("partnerid");

                entity.Property(e => e.Percent).HasColumnName("percentage");

                entity.HasOne(d => d.Partner)
                    .WithMany(p => p.Discount)
                    .HasForeignKey(d => d.Partnerid)
                    .OnDelete(DeleteBehavior.SetNull)
                    .HasConstraintName("discount_partnerid_fkey");
            });
```

Рисунок 11 – Класс ApplicationContext

На следующем этапе разрабатывается сервисный слой, инкапсулирующий бизнес-логику работы с партнёрами и их продажами. Для этого в библиотеке создаётся интерфейс IService, определяющий основные операции:

– пересчёт скидок партнёров (UpdateDiscounts);

– получение партнёра по его свойствам (GetPartnerByProperties);

– загрузка списка партнёров с учётом скидок (LoadPartners, GetPartners);

– добавление, изменение и удаление партнёров (AddPartner, UpdatePartner, DeletePartner);

– получение продаж конкретного партнёра (GetSales);

– добавление, изменение и удаление продаж (AddSale, UpdateSale, DeleteSale);

– сохранение изменений в контексте базы данных (SaveChanges). (См. Рис 12)

```
using ParshakovaLib.Context;
using ParshakovaLib.Models;
using System.Collections.Generic;

namespace ParshakovaLib
{

    public interface IService
    {

        void UpdateDiscounts(ApplicationContext context);


        Partner GetPartnerByProperties(ApplicationContext context, string name);


        void SaveChanges(ApplicationContext context);


        List<Partner> LoadPartners(ApplicationContext context);


        void AddPartner(ApplicationContext context, Partner partner);


        void UpdatePartner(ApplicationContext context, Partner partner);


        IEnumerable<Sale> GetSales(ApplicationContext context, Partner partner);


        void DeletePartner(ApplicationContext context, Partner partner);


        IEnumerable<Partner> GetPartners(ApplicationContext context);


        void AddSale(ApplicationContext context, Sale sale);


        void UpdateSale(ApplicationContext context, Sale sale);


        void DeleteSale(ApplicationContext context, Sale sale);
    }
}
```

Рисунок 12 – Интерфейс IService

Интерфейс IService реализуется в классе Service. В данном классе сосредоточена логика вычисления скидок, работы с коллекциями партнёров и продаж, а также взаимодействия с контекстом баз данных. Одним из ключевых методов является UpdateDiscounts, пересчитывающий скидки партнёров на основе общего объёма реализованной продукции. Метод получает список партнёров вместе с их продажами, суммирует количество проданных единиц и

на основании заранее определённых пороговых значений устанавливает размер скидки. Затем информация о скидках сохраняется в таблице discount. (См. Рис 13)

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Microsoft.EntityFrameworkCore;
using ParshakovaLib.Context;
using ParshakovaLib.Models;

namespace ParshakovaLib
{
    // Сервис работы с партнёрами и продажами

    public class Service : IService
    {
        // Пересчёт скидок партнёров на основе общего объёма продаж

        public void UpdateDiscounts(ApplicationContext _context)
        {
            var partners = _context.Partner
                .Include(p => p.Sale)
                .ToList();
            if (partners != null)
            {
                foreach (var partner in partners)
                {
                    var totalSales = partner.Sale?.Sum(s => s.Quantity) ?? 0;

                    double discountPercentage;

                    if (totalSales <= 10000)
                    {
                        discountPercentage = 0;
                    }
                    else if (totalSales <= 50000)
                    {
                        discountPercentage = 5;
                    }
                    else if (totalSales <= 300000)
                    {
                        discountPercentage = 10;
                    }
                    else
                    {
                        discountPercentage = 15;
                    }

                    var discount = _context.Discount.FirstOrDefault(d => d.Partnerid == partner.Id);
                    if (discount == null)
                    {
                        _context.Discount.Add(new Discount
                        {
                            Partnerid = partner.Id,
                            Percent = discountPercentage
                        });
                    }
                    else
                    {
                        discount.Percent = discountPercentage;
                    }
                }
                _context.SaveChanges();
            }
```

Рисунок 13 – Класс Service

В данном примере демонстрируется типичный подход к работе с Entity Framework Core: извлечение сущностей вместе с навигационными свойствами, применение бизнес-правил к вычислению производных данных (скидок), а также сохранение изменений в контексте. Комментарии в коде поясняют назначение основных шагов метода и делают реализацию более понятной для сопровождения.

После реализации сервисного слоя разрабатывается пользовательский интерфейс на базе технологии WPF.

В отдельном проекте ParshakovaWpf создаётся главное окно MainWindow, в котором отображаются список партнёров и история их продаж. В XAML-разметке главного окна формируется меню с командами для добавления, редактирования и удаления партнёров, а также для управления продажами. Слева располагается список партнёров, представленный элементом ListBox с шаблоном отображения основных сведений: типа организации, наименования, директора, телефона, рейтинга и текущей скидки. Справа размещается DataGrid, показывающий продажи выбранного партнёра: наименование товара, дату продажи и количество. (См. Рис 14-15)

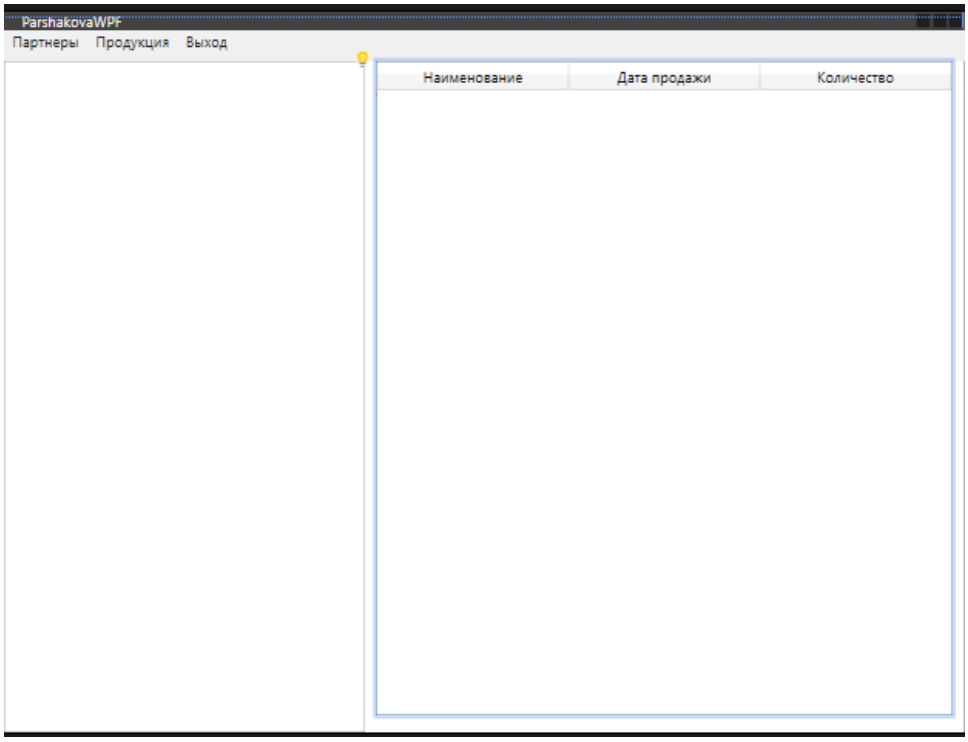

Рисунок 14 – Интерфейс главного окна

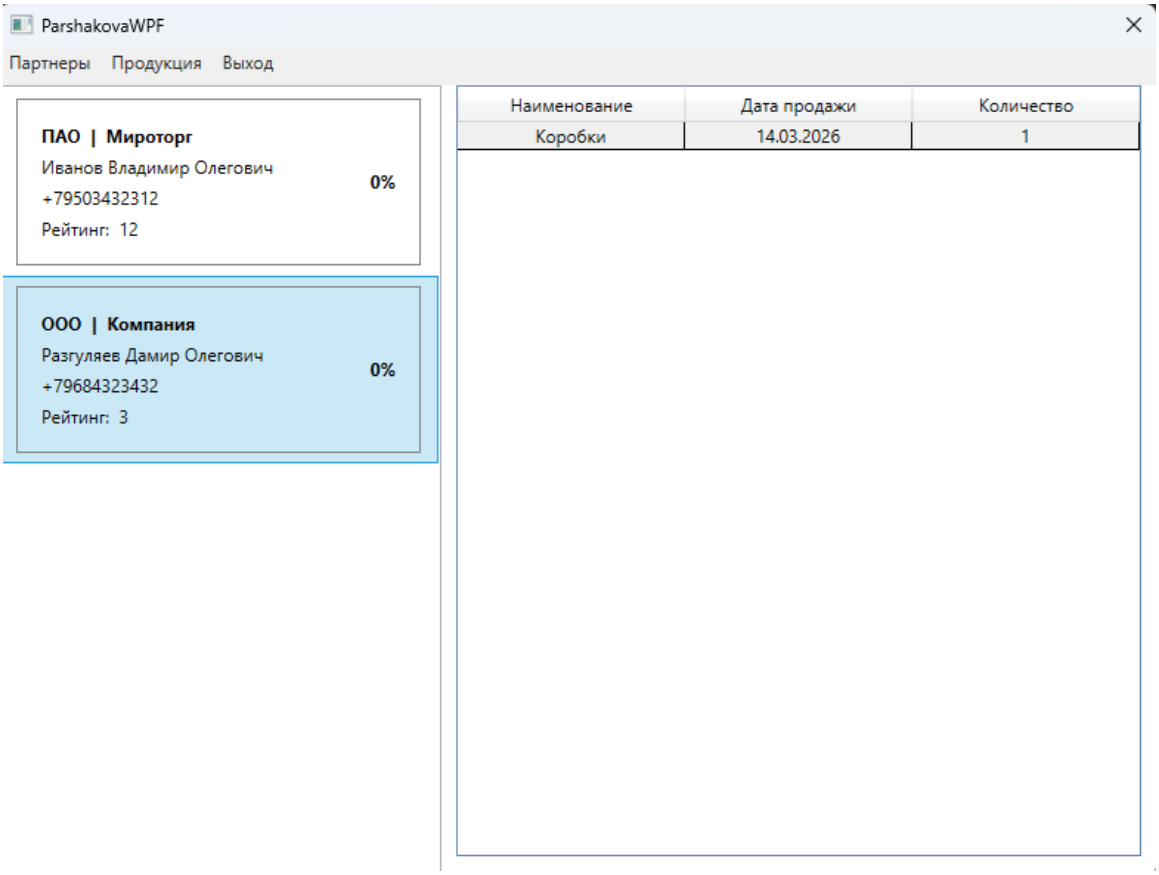

Рисунок 15 – Отображение списка партнёров и продаж

В коде-behind класса MainWindow реализуется логика загрузки данных из библиотеки ParshakovaLib, обновления списков и реакции на действия пользователя.

При инициализации окна создаётся экземпляр контекста ApplicationContext и сервиса Service.

Метод RefreshPartnersView вызывает UpdateDiscounts для актуализации скидок, затем загружает список партнёров с помощью LoadPartners и устанавливает его в качестве источника данных для элемента PartnersList.

Метод RefreshSalesView запрашивает у сервиса GetSales и отображает продажи выбранного партнёра в DataGrid.

Также реализованы обработчики событий AddPartner_Click, EditPartner_Click, DeletePartner_Click, AddSale_Click, EditSale_Click и

DeleteSale_Click, открывающие соответствующие диалоговые окна и обновляющие данные после выполнения операций. (См. Рис 16)

```
namespace ParshakovaWpf
{
    /// <summary>
    /// Interaction logic for MainWindow.xaml
    /// </summary>
    Ссылок: 2
    public partial class MainWindow : Window
    {
        // Основной контекст подключения к базе данных
        private readonly ApplicationContext _databaseContext;
        // Сервис управления партнёрами и продажами
        private Service _partnerManager;
        Ссылок: 0
        public MainWindow()
        {
            InitializeComponent();
            _databaseContext = new ApplicationContext();
            _partnerManager = new Service();
            PartnersList.SelectionChanged += (s, e) => RefreshSalesView();
            RefreshPartnersView();
            RefreshSalesView();
        }

        // Обновление списка партнёров и пересчёт скидок
        Ссылок: 7
        private void RefreshPartnersView()
        {
            var currentSelectedPartner = PartnersList.SelectedItem as Partner;
            _partnerManager.UpdateDiscounts(_databaseContext);
            var partnerCollection = _partnerManager.LoadPartners(_databaseContext);
            PartnersList.ItemsSource = partnerCollection;
            if (currentSelectedPartner != null)
            {
                PartnersList.SelectedItem = partnerCollection.FirstOrDefault(p => p.Name == currentSelectedPartner.Name);
            }
        }

        Ссылок: 1
        private void Exit_Click(object sender, RoutedEventArgs e)
        {
            this.Close();
        }

        // Обновление списка продаж выбранного партнёра
        Ссылок: 5
        private void RefreshSalesView()
        {
            var currentSelectedPartner = PartnersList.SelectedItem as Partner;
            if (currentSelectedPartner != null)
            {
                var partnerData = _partnerManager.GetPartnerByProperties(_databaseContext, currentSelectedPartner.Name);
                if (partnerData != null)
                {
                    SalesDataGrid.ItemsSource = _partnerManager.GetSales(_databaseContext, partnerData);
                }
            }
            else
            {
                SalesDataGrid.ItemsSource = null;
            }
        }
```

Рисунок 16 – Часть кода класса MainWindow

Для операций добавления и редактирования партнёров создаётся отдельное окно PartnerAddEditWindow. В XAML-разметке этого окна размещаются поля для ввода типа партнёра (ComboBox с предопределёнными значениями), наименования, директора, электронной почты, телефона, юридического адреса и рейтинга. Управляющие элементы сопровождаются подписями и расположены в вертикальном StackPanel, а для завершения операции предусмотрены кнопки «Сохранить» и «Отменить». (См. Рис 17)

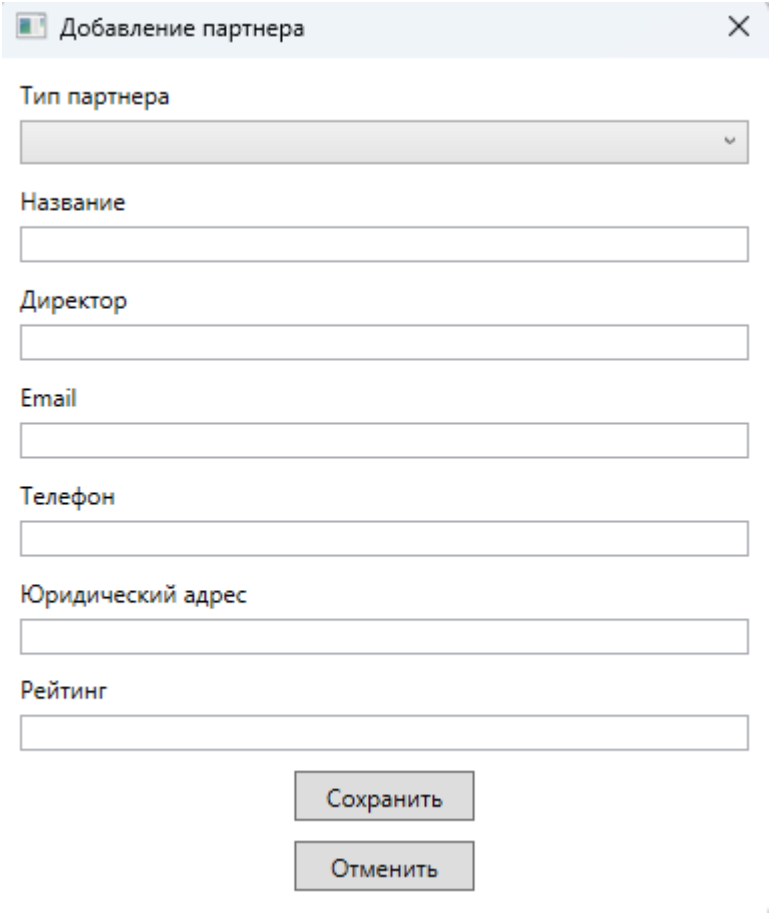

Рисунок 17 – Интерфейс окна добавления и редактирования партнёра

Логика окна PartnerAddEditWindow реализована в одноимённом классе. Конструктор принимает контекст базы данных и, при необходимости, объект редактируемого партнёра. Если партнёр передан, поля формы заполняются его текущими значениями, а заголовок окна меняется на «Редактирование

партнёра». При нажатии кнопки «Сохранить» выполняется поэтапная валидация введённых данных: проверяется заполненность обязательных полей, корректность формата электронной почты и номера телефона, а также то, что рейтинг является положительным целым числом. В случае добавления создаётся новый объект Partner, который передаётся в метод AddPartner сервиса. При редактировании изменяются свойства существующего объекта и вызывается метод UpdatePartner. В случае ошибок пользователю отображаются поясняющие сообщения. (См. Рис 18–19)

```
Ссылок: 6
public partial class PartnerAddEditWindow : Window
{
    // Контекст БД, используемый окном
    private readonly ApplicationContext _databaseContext;
    // Сервис для операций с партнёрами
    private readonly Service _partnerManager;
    // Текущий редактируемый партнёр (или null при добавлении)
    private Partner _currentPartner;

    Ссылок: 2
    public PartnerAddEditWindow(ApplicationContext context, Partner partnerToEdit = null)
    {
        InitializeComponent();
        _databaseContext = context;
        _partnerManager = new Service();
        _currentPartner = partnerToEdit;

        // Предзаполняем поля, если партнёр редактируется
        if (_currentPartner != null)
        {
            Title = "Редактирование партнера";
            TypeComboBox.Text = _currentPartner.Type;
            NameTextBox.Text = _currentPartner.Name;
            DirectorTextBox.Text = _currentPartner.Director;
            EmailTextBox.Text = _currentPartner.Email;
            PhoneTextBox.Text = _currentPartner.Phone;
            LegalAddressTextBox.Text = _currentPartner.Legaladdress;
            RatingTextBox.Text = _currentPartner.Rating.ToString();
        }
        else
        {
            Title = "Добавление партнера";
        }
    }

    // Валидация и сохранение партнёра (добавление/редактирование)
    Ссылок: 1
    private void SaveButton_Click(object sender, RoutedEventArgs e)
    {
        try
        {
            if (string.IsNullOrWhiteSpace(NameTextBox.Text))
            {
                MessageBox.Show("Поле 'Название' не может быть пустым.", "Ошибка ввода", MessageBoxButton.OK, MessageBoxImage.Error);
                return;
            }
            if (string.IsNullOrWhiteSpace(TypeComboBox.Text))
            {
                MessageBox.Show("Поле 'Тип' не может быть пустым.", "Ошибка ввода", MessageBoxButton.OK, MessageBoxImage.Error);
                return;
            }
            if (string.IsNullOrWhiteSpace(DirectorTextBox.Text))
            {
                MessageBox.Show("Поле 'Директор' не может быть пустым.", "Ошибка ввода", MessageBoxButton.OK, MessageBoxImage.Error);
                return;
            }
            if (string.IsNullOrWhiteSpace(EmailTextBox.Text) || !IsValidEmail(EmailTextBox.Text))
            {
                MessageBox.Show("Введите корректный адрес электронной почты (пример: example@mail.com).", "Ошибка ввода", MessageBoxButton.OK, MessageBoxImage.Error);
                return;
```

Рисунок 18 – Часть кода окна PartnerAddEditWindow

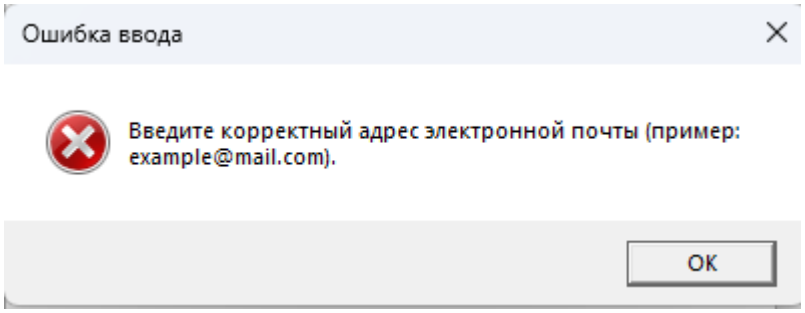

Рисунок 19 – Проверка корректности введённых данных

Аналогично разрабатывается окно SaleAddEditWindow, предназначенное для работы с продажами. В интерфейсе окна размещаются поля для наименования товара, даты продажи и количества. В конструктор передаётся контекст базы данных, информация о текущем партнёре и, при необходимости, редактируемая продажа.

Метод LoadSaleDetails загружает данные существующей продажи в элементы интерфейса. При сохранении продажи выполняется проверка того, что наименование товара не пустое, количество является положительным числом, дата выбрана и не превышает текущую дату. После успешной проверки для новой продажи вызывается метод AddSale, а для существующей – UpdateSale. Все изменения сохраняются в базе данных, а окно закрывается с положительным результатом. (См. Рис 20-21)

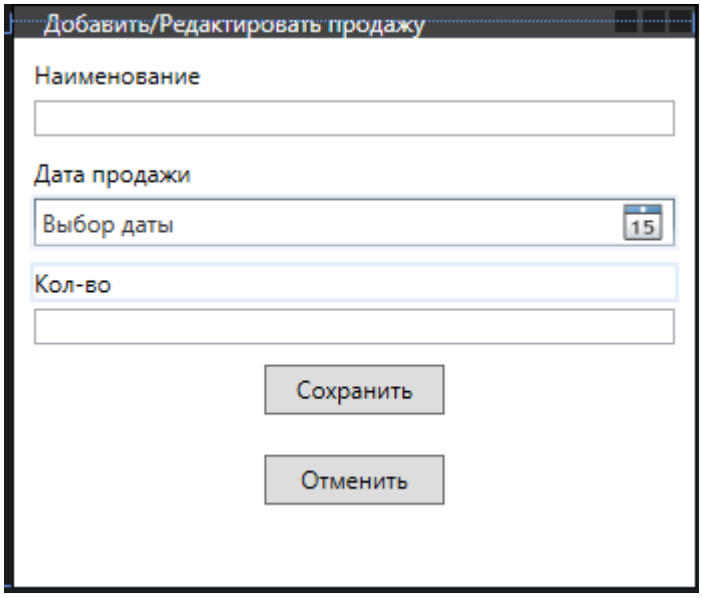

Рисунок 20 – Интерфейс окна добавления и редактирования продажи

```
Ссылок 1
private void SaveButton_Click(object sender, RoutedEventArgs e)
{
    try
    {
        // Базовая валидация введённых данных
        if (string.IsNullOrWhiteSpace(ProductNameTextBox.Text))
        {
            MessageBox.Show("Введите корректное наименование продукта.", "Ошибка ввода", MessageBoxButton.OK, MessageBoxImage.Error);
            return;
        }

        if (!int.TryParse(QuantityTextBox.Text, out int saleQuantity) || saleQuantity <= 0)
        {
            MessageBox.Show("Количество должно быть положительным числом.", "Ошибка ввода", MessageBoxButton.OK, MessageBoxImage.Error);
            return;
        }

        if (SaleDatePicker.SelectedDate == null)
        {
            MessageBox.Show("Выберите дату продажи.", "Ошибка ввода", MessageBoxButton.OK, MessageBoxImage.Error);
            return;
        }

        if (SaleDatePicker.SelectedDate > DateTime.Today)
        {
            MessageBox.Show("Дата продажи не может быть в будущем.", "Ошибка ввода", MessageBoxButton.OK, MessageBoxImage.Error);
            return;
        }

        // Сохранение изменений продажи
        _currentSale.ProductName = ProductNameTextBox.Text;
        _currentSale.Quantity = saleQuantity;
        _currentSale.Date = SaleDatePicker.SelectedDate.Value;
        _currentSale.Partnerid = _currentPartner.Id;

        if (_isEditing)
        {
            _partnerManager.UpdateSale(_databaseContext, _currentSale);
        }
        else
        {
            _partnerManager.AddSale(_databaseContext, _currentSale);
        }

        _partnerManager.SaveChanges(_databaseContext);
        DialogResult = true;
        Close();
    }
    catch (Exception error)
    {
        MessageBox.Show($"Произошла ошибка: {error.Message}", "Ошибка", MessageBoxButton.OK, MessageBoxImage.Error);
    }
}
```

Рисунок 21 – Логика обработки данных продажи

Откладка приложения

В результате при запуске приложения пользователь видит список партнёров, загружаемый из базы данных parshakova_db, а также историю продаж выбранного партнёра. При добавлении нового партнёра через главное меню или контекстное меню открывается окно PartnerAddEditWindow.

После корректного заполнения всех обязательных полей и успешного прохождения валидации данные сохраняются в базе, и новый партнёр появляется в списке. При добавлении продажи через меню или контекстное меню DataGrid открывается окно SaleAddEditWindow, в котором указываются наименование товара, дата и количество. После сохранения данные о продаже

фиксируются в таблице sale, а метод UpdateDiscounts пересчитывает скидку партнёра, учитывая суммарный объём продаж. (См. Рис 22–25)

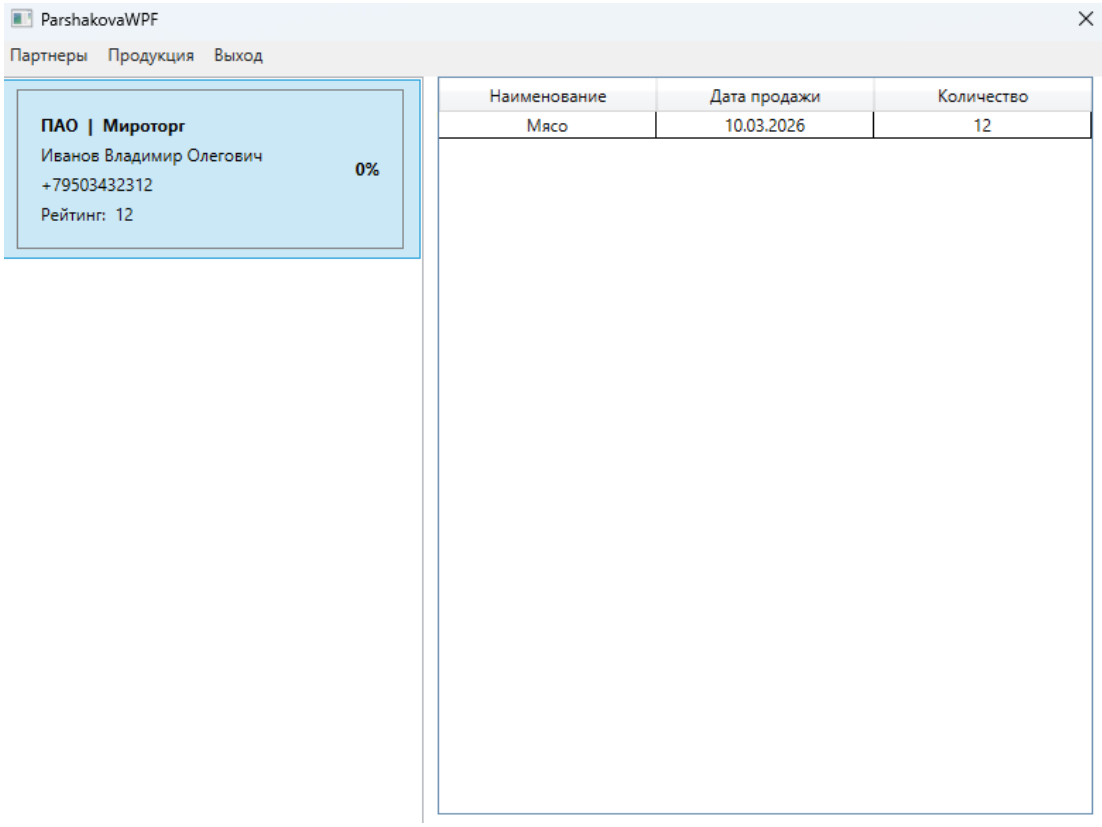

Рисунок 22 – Главное окно с загруженными данными

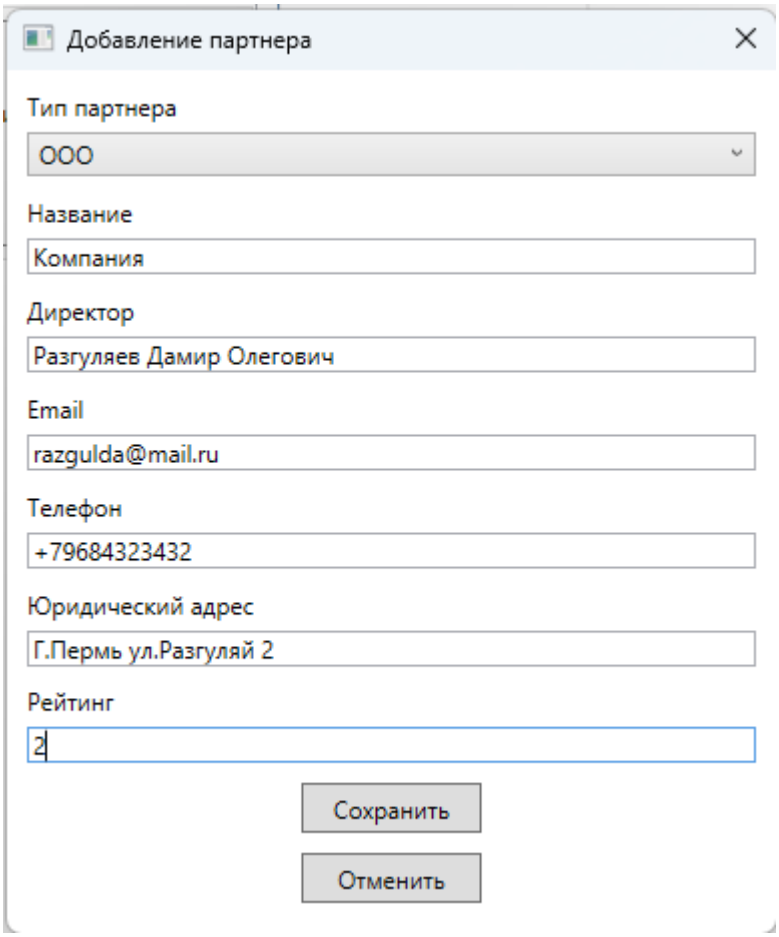

Рисунок 23 – Добавление нового партнёра

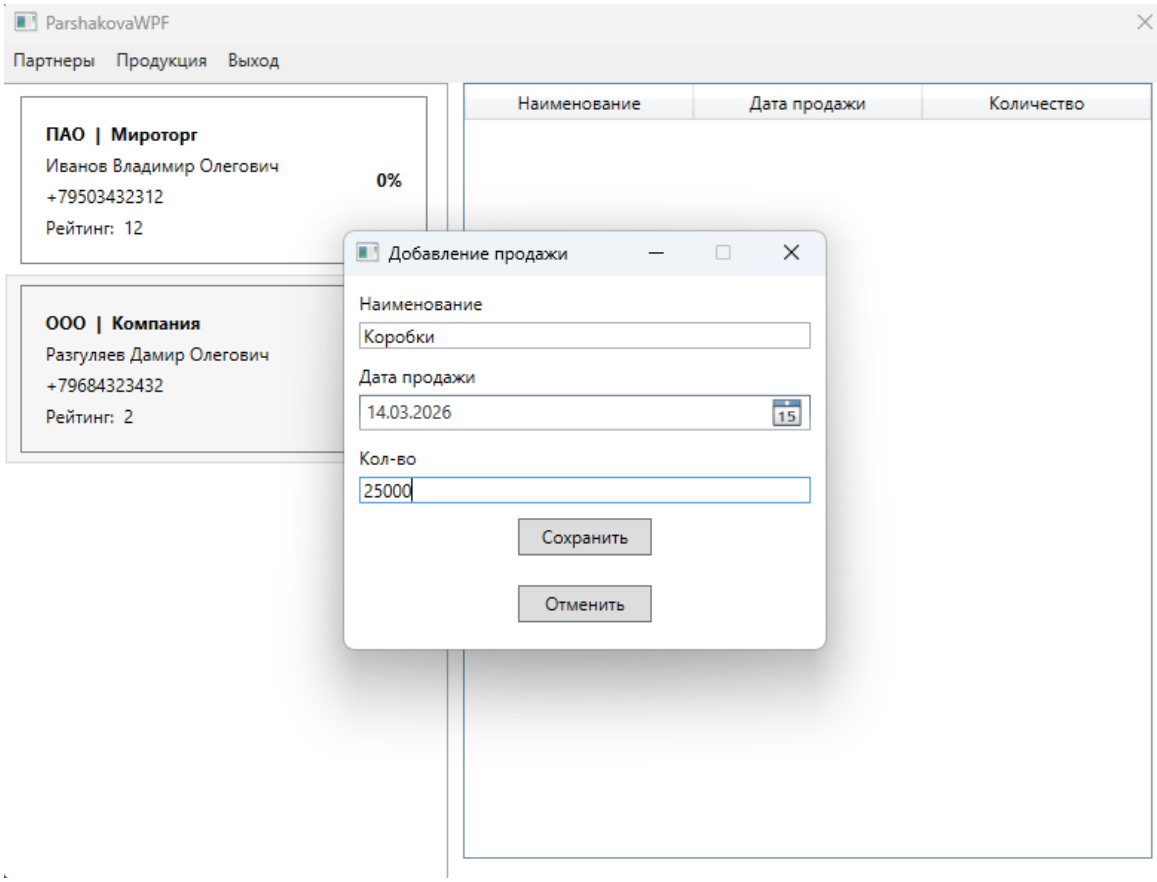

Рисунок 24 – Добавление новой продажи

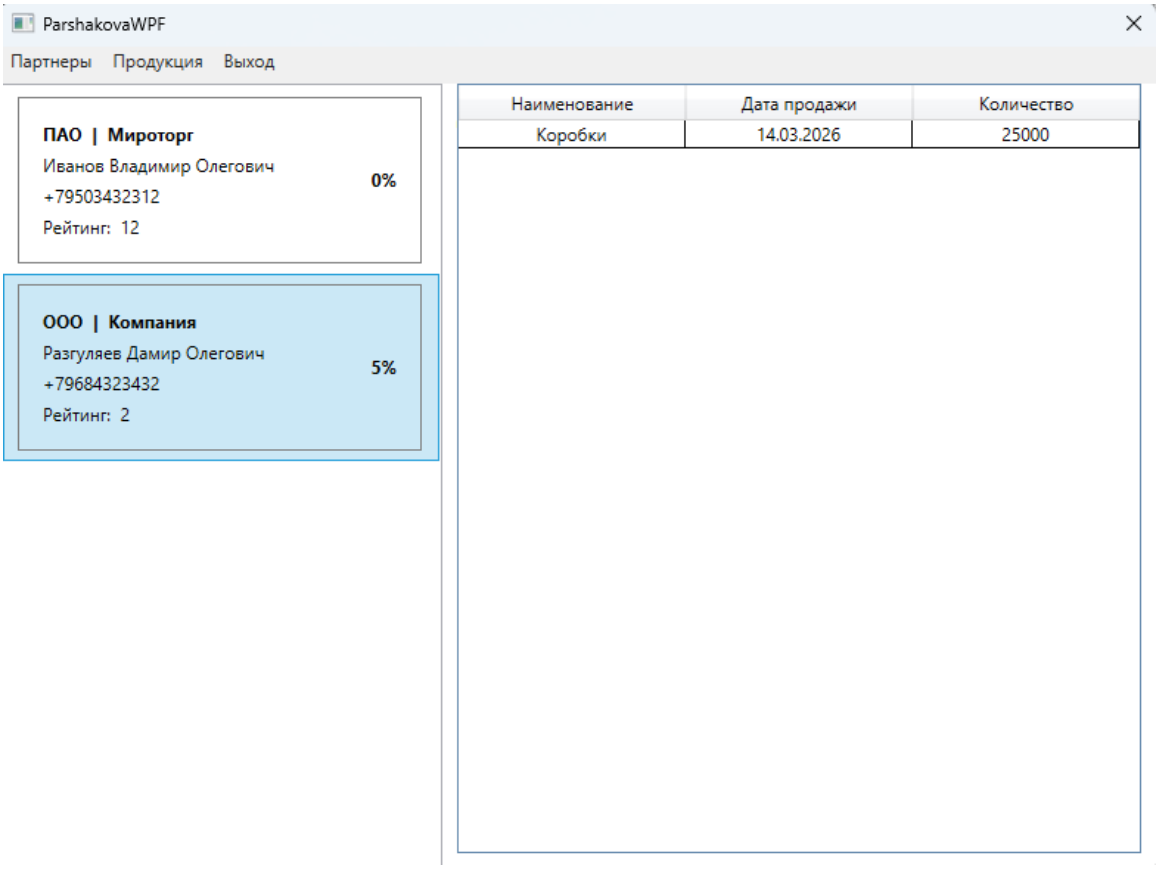

Рисунок 25 – Обновление скидки после добавления продаж

Кроме того, в приложении реализована возможность редактирования ранее введённых сведений. При выборе команды «Редактировать партнёра» открывается окно PartnerAddEditWindow с предзаполненными полями. После внесения изменений и успешной проверки данные партнёра обновляются в базе. Аналогичный механизм используется для изменения параметров продаж: при выборе записи в таблице и вызове команды редактирования открывается окно SaleAddEditWindow, позволяющее скорректировать наименование, дату и количество. После сохранения обновлённые данные отображаются в интерфейсе, а скидка пересчитывается при необходимости. (См. Рис 26–28)

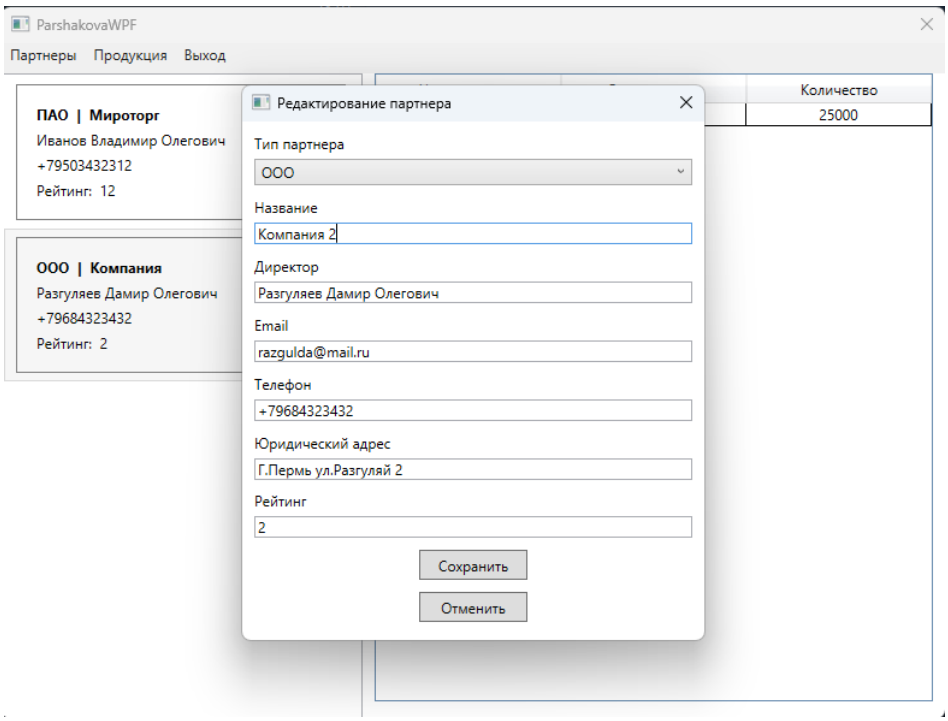

Рисунок 26 – Редактирование данных партнёра

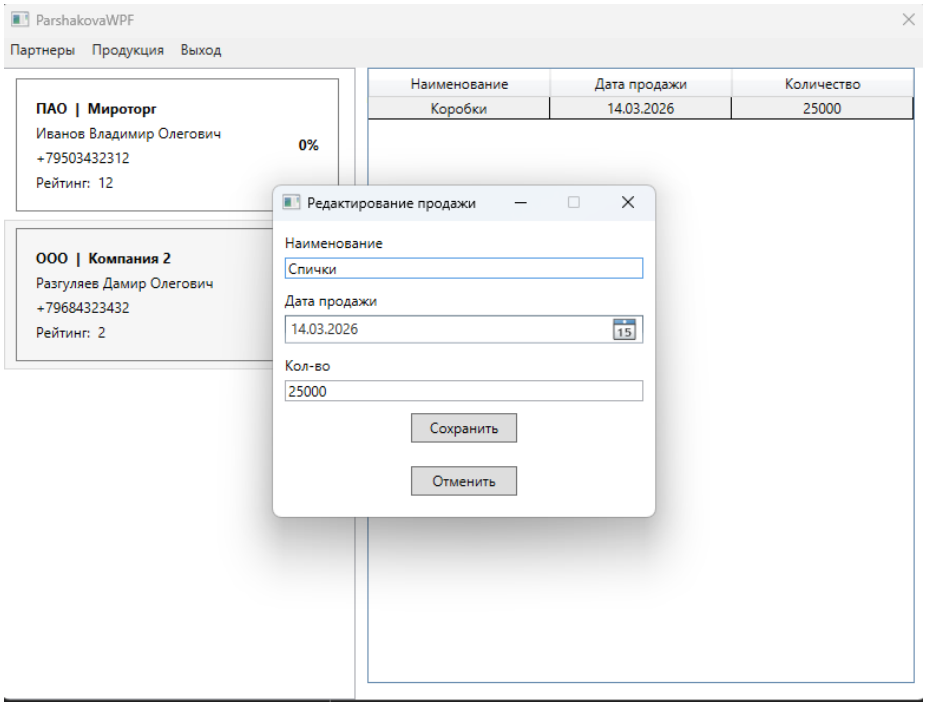

Рисунок 27 – Редактирование продажи

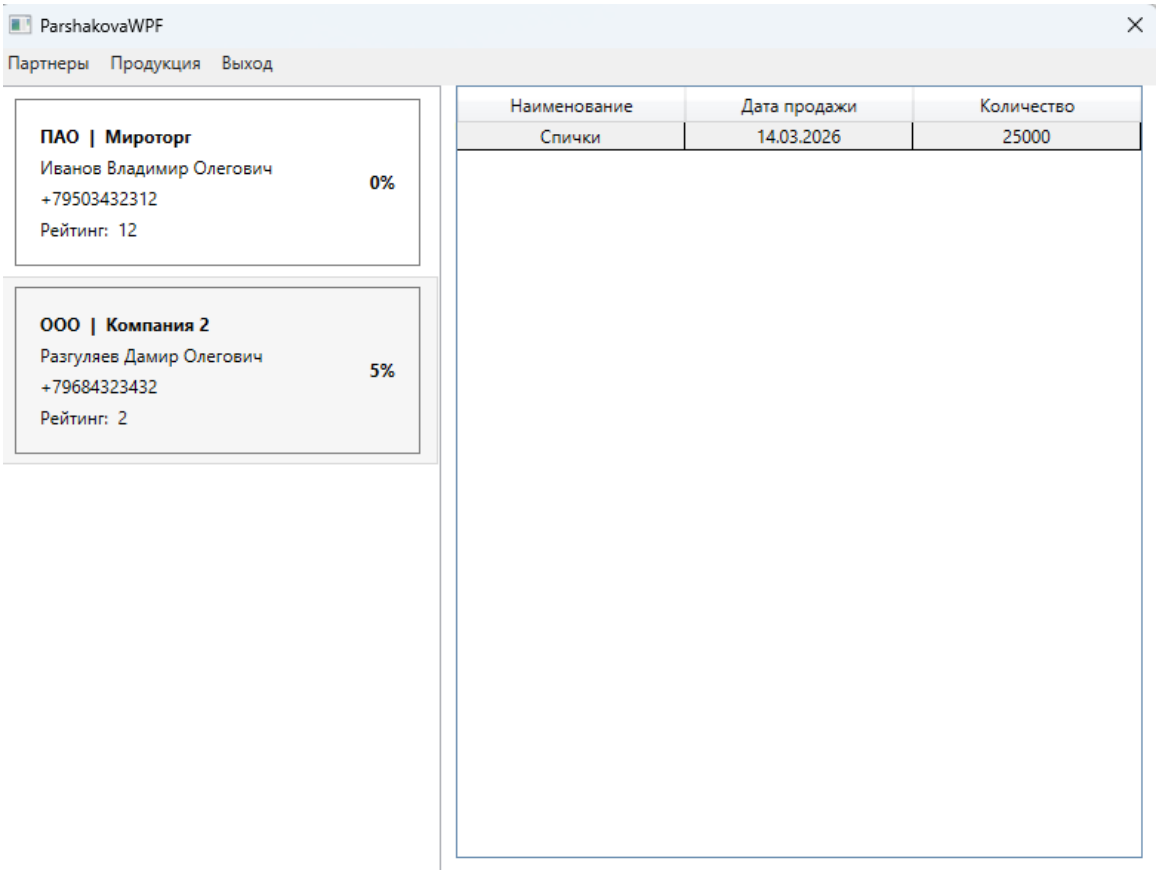

Рисунок 28 – Обновлённые сведения о партнёре и продаже

Для поддержания корректности данных также предусмотрены операции удаления. При попытке удалить партнёра система запрашивает подтверждение действия. В случае согласия вызывается метод DeletePartner сервиса, который удаляет не только самого партнёра, но и связанные с ним продажи. Аналогично при удалении отдельной продажи пользователь подтверждает действие, после чего запись удаляется из таблицы sale, а данные в интерфейсе обновляются. Такой подход позволяет избежать случайной потери информации и повышает надёжность работы приложения. (См. Рис 29–32)

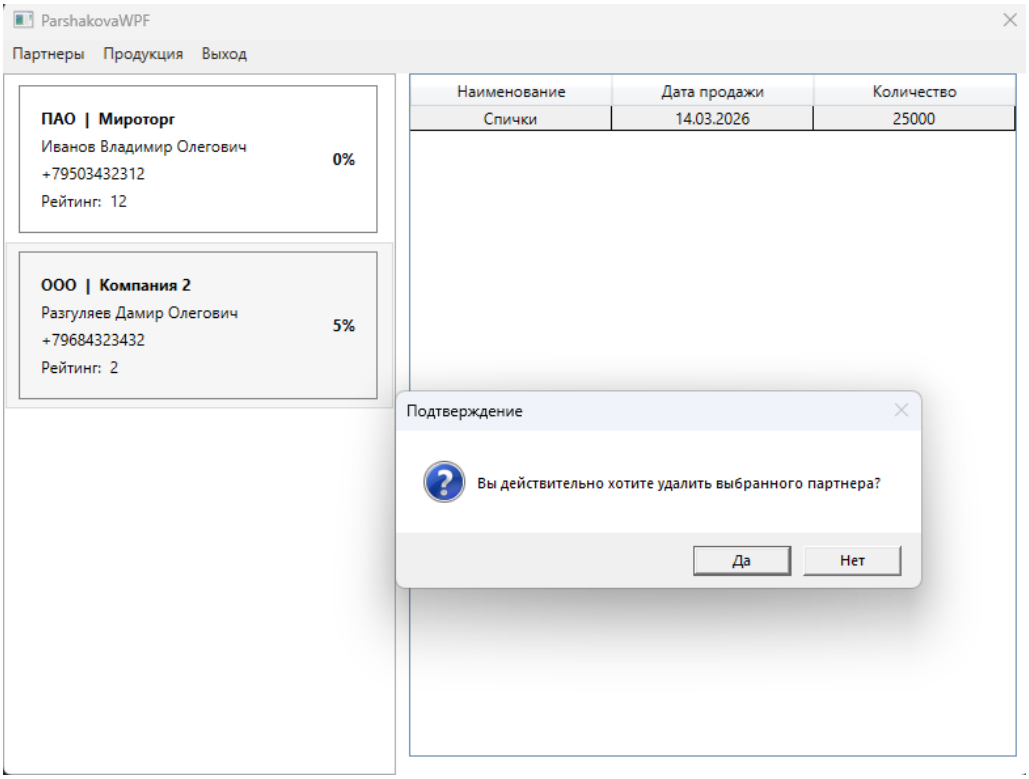

Рисунок 29 – Подтверждение удаления партнёра

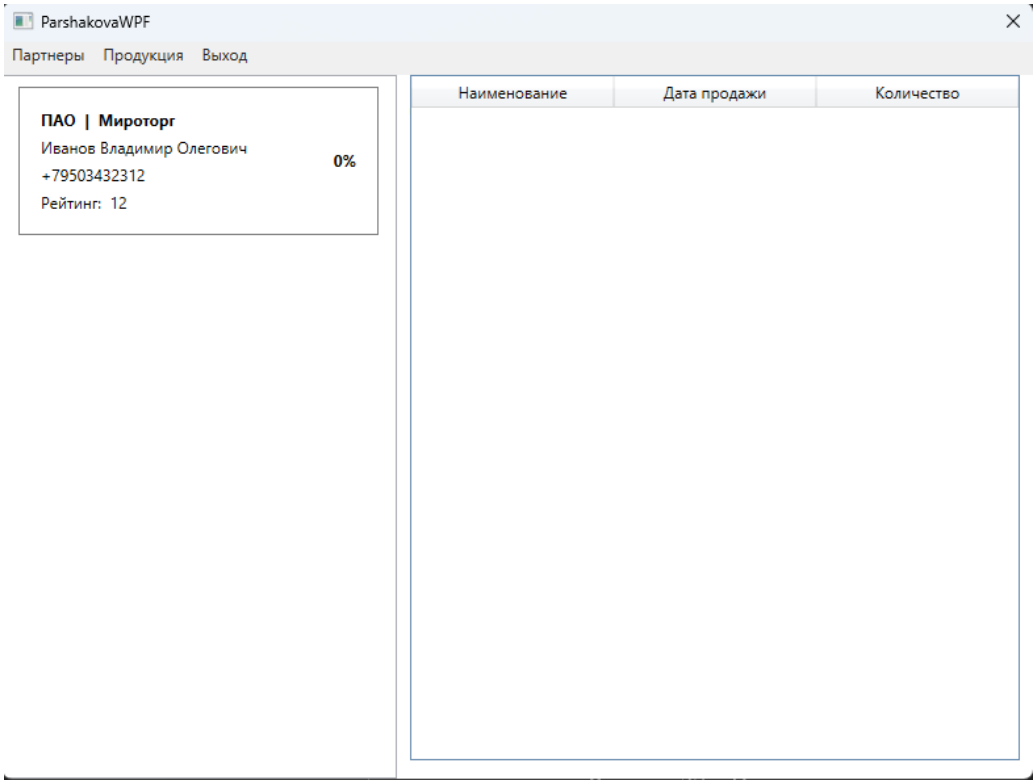

Рисунок 30 – Удаление выбранного партнёра

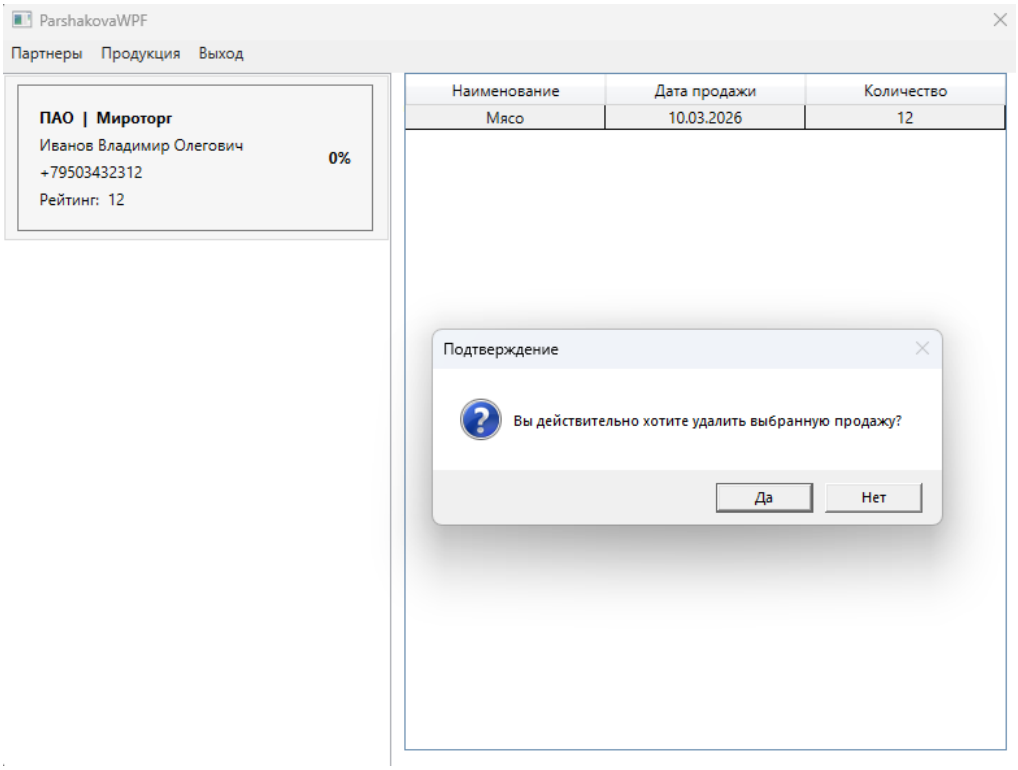

Рисунок 31 – Подтверждение удаления продажи

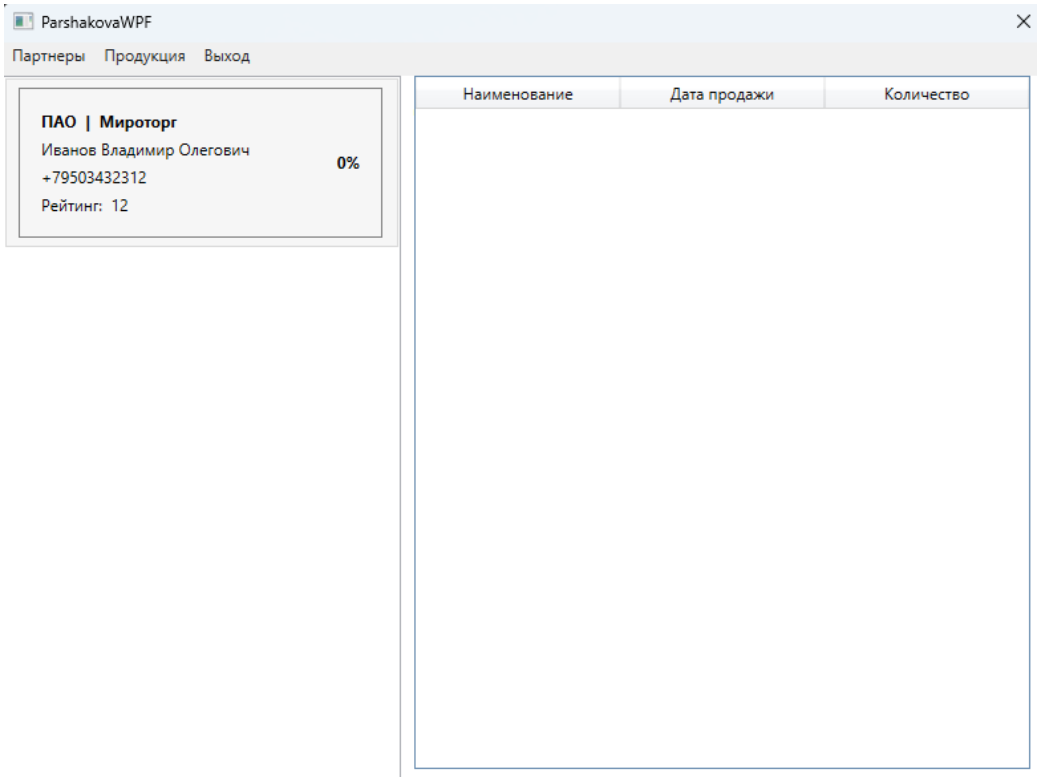

Рисунок 32 – Удаление выбранной продажи

Процесс тестирования

Для проверки корректности работы реализованной функциональности используется модульное (unit) тестирование. Такой подход позволяет проверить отдельные методы сервиса и убедиться, что операции с базой данных выполняются в соответствии с требованиями. Тесты создаются в отдельном проекте ParshakovaTests, реализованном с использованием фреймворка MSTest.

В тестовом проекте определяется класс ParshakovaTests, содержащий набор методов с атрибутом TestMethod. Для доступа к базе данных тесты используют реальный контекст ApplicationContext, настроенный на ту же базу данных parshakova_db. Вспомогательный метод CleanupTestData удаляет временные данные, созданные в процессе тестирования: он находит партнёров, имя которых начинается с префикса «Test_», соответствующие им продажи и скидки, а затем удаляет все найденные записи. Такой подход позволяет поддерживать чистоту тестовой среды и повторно запускать тесты без накопления служебных данных. (См. Рис 33)

```
private void CleanupTestData(ApplicationContext context)
{
    var testPartners = context.Partner
        .Where(p => p.Name.StartsWith("Test_"))
        .ToList();

    if (testPartners.Count == 0)
        return;

    var partnerIds = testPartners.Select(p => p.Id).ToList();

    var sales = context.Sale
        .Where(s => s.Partnerid != null && partnerIds.Contains(s.Partnerid.Value))
        .ToList();
    if (sales.Count > 0)
    {
        context.Sale.RemoveRange(sales);
    }

    var discounts = context.Discount
        .Where(d => d.Partnerid != null && partnerIds.Contains(d.Partnerid.Value))
        .ToList();
    if (discounts.Count > 0)
    {
        context.Discount.RemoveRange(discounts);
    }

    context.Partner.RemoveRange(testPartners);
    context.SaveChanges();
}
```

Рисунок 33 – Вспомогательный метод очистки тестовых данных

Один из важных тестов UpdateDiscounts_SetsCorrectDiscount_ForDifferentTotalSales – проверяет корректность работы метода UpdateDiscounts. В нём последовательно создаются

несколько тестовых партнёров с различным суммарным количеством продаж. Для каждого партнёра в таблицу sale добавляется запись с определённым значением Quantity. Затем вызывается метод UpdateDiscounts, после чего из таблицы discount извлекаются рассчитанные значения скидок. С помощью CollectionAssert.AreEqual проверяется, что полученные значения соответствуют ожидаемым порогам (0, 5, 10 и 15 процентов). (См. Рис 34)

```
Ссылок: 0
public void UpdateDiscounts_SetsCorrectDiscount_ForDifferentTotalSales()
{
    using var context = CreateContext();
    CleanupTestData(context);
    try
    {
        var partners = new[]
        {
        new Partner
        {
            Name = "Test_P1",
            Type = "T",
            Director = "D",
            Email = "e1@test",
            Phone = "1",
            Legaladdress = "addr",
            Rating = 1,
            Sale = new List<Sale> { new() { Quantity = 1000, ProductName = "Test_Prod", Date = DateTime.Today } }
        },
        new Partner
        {
            Name = "Test_P2",
            Type = "T",
            Director = "D",
            Email = "e2@test",
            Phone = "2",
            Legaladdress = "addr",
            Rating = 1,
            Sale = new List<Sale> { new() { Quantity = 20000, ProductName = "Test_Prod", Date = DateTime.Today } }
        },
        new Partner
        {
            Name = "Test_P3",
            Type = "T",
            Director = "D",
            Email = "e3@test",
            Phone = "3",
            Legaladdress = "addr",
            Rating = 1,
            Sale = new List<Sale> { new() { Quantity = 100000, ProductName = "Test_Prod", Date = DateTime.Today } }
        },
        new Partner
        {
            Name = "Test_P4",
            Type = "T",
            Director = "D",
            Email = "e4@test",
            Phone = "4",
            Legaladdress = "addr",
            Rating = 1,
            Sale = new List<Sale> { new() { Quantity = 400000, ProductName = "Test_Prod", Date = DateTime.Today } }
        },
    };

        context.Partner.AddRange(partners);
        context.SaveChanges();

        var service = new Service();

        service.UpdateDiscounts(context);

        var discounts = context.Discount
            .Where(d => d.Partner != null && d.Partner.Name.StartsWith("Test_"))
            .OrderBy(d => d.Partnerid)
            .Select(d => d.Percentage ?? 0)
            .ToList();

        CollectionAssert.AreEqual(new List<double> { 0, 5, 10, 15 }, discounts);
    }
    finally
    {
        CleanupTestData(context);
    }
}
```

Рисунок 34 – Тест метода UpdateDiscounts

Дополнительный тест GetPartnerByProperties_ReturnsPartnerByName проверяет корректность поиска партнёра по имени. В рамках теста в базу

данных добавляется партнёр с уникальным именем «Test_Partner». После сохранения контекста создаётся экземпляр сервиса Service и вызывается метод GetPartnerByProperties с этим именем. С помощью Assert.IsNotNull и Assert.AreEqual удостоверяются факт нахождения партнёра и корректность возвращаемых данных. После завершения теста временные записи удаляются. (См. Рис 35)

```
[TestMethod]

public void GetPartnerByProperties_ReturnsPartnerByName()
{
    using var context = CreateContext();
    CleanupTestData(context);
    try
    {
        context.Partner.Add(new Partner
        {
            Name = "Test_Partner",
            Type = "T",
            Director = "D",
            Email = "e@test",
            Phone = "1",
            Legaladdress = "addr",
            Rating = 1
        });
        context.SaveChanges();

        var service = new Service();

        var result = service.GetPartnerByProperties(context, "Test_Partner");

        Assert.IsNotNull(result);
        Assert.AreEqual("Test_Partner", result.Name);
    }
    finally
    {
        CleanupTestData(context);
    }
}
```

Рисунок 35 – Тест метода GetPartnerByProperties

Следующая группа тестов посвящена операциям добавления, обновления и удаления партнёров. В тесте AddPartner_AddsNewPartner создаётся объект Partner с тестовыми данными и передаётся в метод AddPartner сервиса. Затем проверяется, что в таблице partner появилась запись с указанным именем. Тест UpdatePartner_UpdatesExistingPartner сначала добавляет партнёра в базу, затем изменяет его имя и вызывает метод UpdatePartner. После этого выполняется

запрос к базе данных, подтверждающий, что изменения зафиксированы. Наконец, тест DeletePartner_DeletesPartnerAndSales проверяет, что при удалении партнёра удаляются связанные с ним продажи: после вызова DeletePartner в таблицах partner и sale не должно оставаться записей с тестовыми значениями. (См. Рис 36–37)

```
[TestMethod]
Ссылок: 0
public void AddPartner_AddsNewPartner()
{
    using var context = CreateContext();
    CleanupTestData(context);
    try
    {
        var service = new Service();
        var partner = new Partner
        {
            Name = "Test_AddPartner",
            Type = "T",
            Director = "D",
            Email = "e@test",
            Phone = "1",
            Legaladdress = "addr",
            Rating = 1
        };

        service.AddPartner(context, partner);

        Assert.IsTrue(context.Partner.Any(p => p.Name == "Test_AddPartner"));
    }
    finally
    {
        CleanupTestData(context);
    }
}

[TestMethod]
Ссылок: 0
public void UpdatePartner_UpdatesExistingPartner()
{
    using var context = CreateContext();
    CleanupTestData(context);
    try
    {
        var partner = new Partner
        {
            Name = "Test_UpdatePartner",
            Type = "T",
            Director = "D",
            Email = "e@test",
            Phone = "1",
            Legaladdress = "addr",
            Rating = 1
        };
        context.Partner.Add(partner);
        context.SaveChanges();

        var service = new Service();
        partner.Name = "Test_UpdatePartner_Changed";

        service.UpdatePartner(context, partner);

        Assert.IsTrue(context.Partner.Any(p => p.Name == "Test_UpdatePartner_Changed"));
    }
    finally
    {
        CleanupTestData(context);
    }
}
```

Рисунок 36 – Тесты методов AddPartner и UpdatePartner

```
[TestMethod]
Ссылок: 0
public void DeletePartner_DeletesPartnerAndSales()
{
    using var context = CreateContext();
    CleanupTestData(context);
    try
    {
        var partner = new Partner
        {
            Name = "Test_DeletePartner",
            Type = "T",
            Director = "D",
            Email = "e@test",
            Phone = "1",
            Legaladdress = "addr",
            Rating = 1
        };
        context.Partner.Add(partner);
        context.SaveChanges();

        var sale = new Sale
        {
            Partnerid = partner.Id,
            ProductName = "Test_DeleteSale",
            Quantity = 5,
            Date = DateTime.Today
        };
        context.Sale.Add(sale);
        context.SaveChanges();

        var service = new Service();

        service.DeletePartner(context, partner);

        Assert.IsFalse(context.Partner.Any(p => p.Name == "Test_DeletePartner"));
        Assert.IsFalse(context.Sale.Any(s => s.ProductName == "Test_DeleteSale"));
    }
    finally
    {
        CleanupTestData(context);
    }
}
```

Рисунок 37 – Тест метода DeletePartner

Отдельный блок тестов проверяет операции с продажами. Тест AddSale_AddsNewSale создаёт тестового партнёра, затем формирует объект Sale и передаёт его в метод AddSale. После выполнения операции проверяется наличие соответствующей записи в таблице sale. В тесте UpdateSale_UpdatesExistingSale добавляется продажа, затем изменяется её количество, вызывается UpdateSale и контролируется, что обновлённое значение Quantity сохранено в базе. Тест DeleteSale_DeletesExistingSale убеждается, что после вызова DeleteSale запись о тестовой продаже отсутствует в таблице sale. (См. Рис 38-39)

```
[TestMethod]
Ссылок: 0
public void AddSale_AddsNewSale()
{
    using var context = CreateContext();
    CleanupTestData(context);
    try
    {
        var partner = new Partner
        {
            Name = "Test_AddSalePartner",
            Type = "T",
            Director = "D",
            Email = "e@test",
            Phone = "1",
            Legaladdress = "addr",
            Rating = 1
        };
        context.Partner.Add(partner);
        context.SaveChanges();

        var service = new Service();

        var sale = new Sale
        {
            Partnerid = partner.Id,
            ProductName = "Test_AddSale",
            Quantity = 10,
            Date = DateTime.Today
        };

        service.AddSale(context, sale);

        Assert.IsTrue(context.Sale.Any(s => s.ProductName == "Test_AddSale"));
    }
    finally
    {
        CleanupTestData(context);
    }
}

[TestMethod]
Ссылок: 0
public void UpdateSale_UpdatesExistingSale()
{
    using var context = CreateContext();
    CleanupTestData(context);
    try
    {
        var partner = new Partner
        {
            Name = "Test_UpdateSalePartner",
            Type = "T",
            Director = "D",
            Email = "e@test",
            Phone = "1",
            Legaladdress = "addr",
            Rating = 1
        };
        context.Partner.Add(partner);
        context.SaveChanges();

        var sale = new Sale
        {
            Partnerid = partner.Id,
            ProductName = "Test_UpdateSale",
            Quantity = 10,
            Date = DateTime.Today
        };
        context.Sale.Add(sale);
        context.SaveChanges();

        var service = new Service();
        sale.Quantity = 20;

        service.UpdateSale(context, sale);

        Assert.AreEqual(20, context.Sale.Single(s => s.ProductName == "Test_UpdateSale").Quantity);
    }
    finally
    {
        CleanupTestData(context);
    }
}
```

Рисунок 38 – Тесты методов AddSale и UpdateSale

```
[TestMethod]
Ссылок: 0
public void DeleteSale_DeletesExistingSale()
{
    using var context = CreateContext();
    CleanupTestData(context);
    try
    {
        var partner = new Partner
        {
            Name = "Test_DeleteSalePartner",
            Type = "T",
            Director = "D",
            Email = "e@test",
            Phone = "1",
            Legaladdress = "addr",
            Rating = 1
        };
        context.Partner.Add(partner);
        context.SaveChanges();

        var sale = new Sale
        {
            Partnerid = partner.Id,
            ProductName = "Test_DeleteSale",
            Quantity = 10,
            Date = DateTime.Today
        };
        context.Sale.Add(sale);
        context.SaveChanges();

        var service = new Service();

        service.DeleteSale(context, sale);

        Assert.IsFalse(context.Sale.Any(s => s.ProductName == "Test_DeleteSale"));
    }
    finally
    {
        CleanupTestData(context);
    }
}
```

Рисунок 39 – Тест метода DeleteSale

Тест GetSales_ReturnsSalesForPartner проверяет корректность выборки продаж для конкретного партнёра. В базу данных добавляются два тестовых партнёра и несколько продаж, причём часть из них относится к первому партнёру, а часть – ко второму. После вызова метода GetSales для первого партнёра с помощью Assert.AreEqual и CollectionAssert.AreEquivalent проверяется, что возвращается именно ожидаемый набор продаж, привязанный к нужному партнёру. (См. Рис 40)

```
[TestMethod]
Ссылок: 0
public void GetSales_ReturnsSalesForPartner()
{
    using var context = CreateContext();
    CleanupTestData(context);
    try
    {
        var partner1 = new Partner
        {
            Name = "Test_P1",
            Type = "T",
            Director = "D",
            Email = "e1@test",
            Phone = "1",
            Legaladdress = "addr",
            Rating = 1
        };
        var partner2 = new Partner
        {
            Name = "Test_P2",
            Type = "T",
            Director = "D",
            Email = "e2@test",
            Phone = "2",
            Legaladdress = "addr",
            Rating = 1
        };

        context.Partner.AddRange(partner1, partner2);
        context.SaveChanges();

        context.Sale.AddRange(
            new Sale { Partnerid = partner1.Id, ProductName = "Test_A", Quantity = 1, Date = DateTime.Today },
            new Sale { Partnerid = partner1.Id, ProductName = "Test_B", Quantity = 2, Date = DateTime.Today },
            new Sale { Partnerid = partner2.Id, ProductName = "Test_C", Quantity = 3, Date = DateTime.Today }
        );
        context.SaveChanges();

        var service = new Service();

        var salesForP1 = service.GetSales(context, partner1).ToList();

        Assert.AreEqual(2, salesForP1.Count);
        CollectionAssert.AreEquivalent(new[] { "Test_A", "Test_B" }, salesForP1.Select(s => s.ProductName).ToList());
    }
    finally
    {
        CleanupTestData(context);
    }
}
```

Рисунок 40 – Тест метода GetSales

Методы GetPartners и LoadPartners, возвращающие партнёров вместе с их скидками, также покрываются отдельными тестами. В них создаётся партнёр с записью о скидке в таблице discount, после чего вызываются соответствующие методы сервиса, и проверяется, что к объекту партнёра загружена коллекция Discount с хотя бы одним элементом. Это подтверждает корректность настройки навигационных свойств и использования Include при выборке данных. (См. Рис 41)

```
[TestMethod]
Ссылок: 0
public void GetPartners_ReturnsPartnersWithDiscounts()
{
    using var context = CreateContext();
    CleanupTestData(context);
    try
    {
        var partner = new Partner
        {
            Name = "Test_GetPartners",
            Type = "T",
            Director = "D",
            Email = "e@test",
            Phone = "1",
            Legaladdress = "addr",
            Rating = 1
        };
        context.Partner.Add(partner);
        context.SaveChanges();

        context.Discount.Add(new Discount
        {
            Partnerid = partner.Id,
            Percentage = 5
        });
        context.SaveChanges();

        var service = new Service();

        var partners = service.GetPartners(context)
            .Where(p => p.Name == "Test_GetPartners")
            .ToList();

        Assert.AreEqual(1, partners.Count);
        Assert.IsTrue(partners[0].Discount.Any());
    }
    finally
    {
        CleanupTestData(context);
    }
}

[TestMethod]
Ссылок: 0
public void LoadPartners_ReturnsPartnersWithDiscounts()
{
    using var context = CreateContext();
    CleanupTestData(context);
    try
    {
        var partner = new Partner
        {
            Name = "Test_LoadPartners",
            Type = "T",
            Director = "D",
            Email = "e@test",
            Phone = "1",
            Legaladdress = "addr",
            Rating = 1
        };
        context.Partner.Add(partner);
        context.SaveChanges();

        context.Discount.Add(new Discount
        {
            Partnerid = partner.Id,
            Percentage = 10
        });
        context.SaveChanges();

        var service = new Service();

        var partners = service.LoadPartners(context)
            .Where(p => p.Name == "Test_LoadPartners")
            .ToList();

        Assert.AreEqual(1, partners.Count);
        Assert.IsTrue(partners[0].Discount.Any());
    }
    finally
    {
        CleanupTestData(context);
    }
}
```

Рисунок 41 – Тесты методов GetPartners и LoadPartners

После написания и запуска всех модульных тестов было зафиксировано их успешное прохождение. Это свидетельствует о корректной реализации основных операций по работе с партнёрами и продажами, а также о правильном взаимодействии приложения с базой данных. Модульное тестирование позволило обнаружить и устранить возможные ошибки на ранних этапах разработки и обеспечило необходимый уровень надёжности программного продукта. (См. Рис 49)

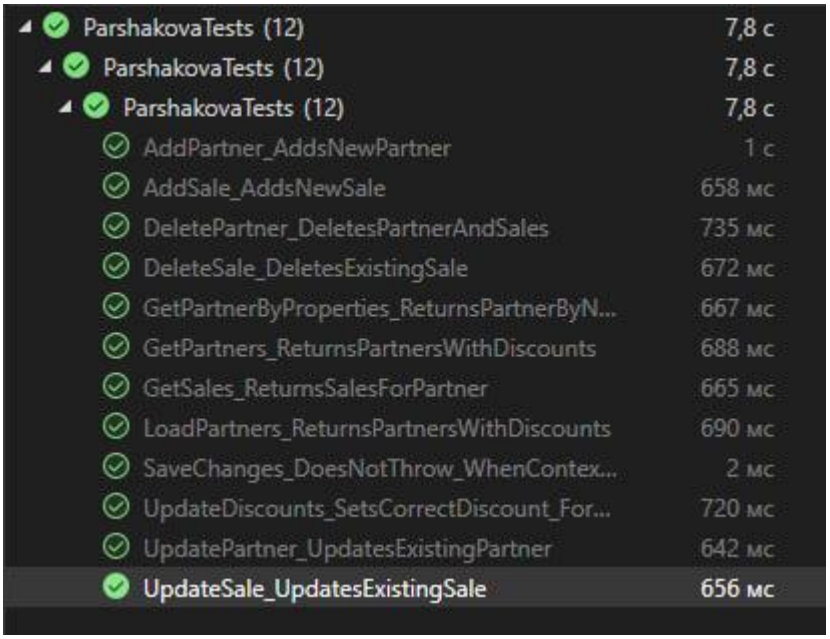

Рисунок 49 – Запуск и успешное выполнение модульных тестов

3.1.3. Использование методов для получения кода с заданной функциональностью и степенью качества

В рамках проекта по учёту партнёров и их продаж код с заданной функциональностью и требуемым качеством получен за счёт системного применения методов анализа требований, проектирования архитектуры и поэтапной реализации.

На начальном этапе были сформулированы функциональные требования к системе: необходимость хранения данных о партнёрах, учёта продаж, расчёта скидок, а также отображения этой информации в удобном графическом интерфейсе. Требования были разделены на уровни: работа с базой данных

PostgreSQL, уровень бизнес-логики на основе библиотеки ParshakovaLib и уровень пользовательского интерфейса WPF-приложения ParshakovaWpf.

Для обеспечения необходимой функциональности были применены методы объектно-ориентированного проектирования. В библиотеке ParshakovaLib выделены отдельные сущности для партнёров, продаж и скидок, а также контекст доступа к данным. Бизнес-логика сосредоточена в сервисе, который инкапсулирует операции добавления, обновления и удаления записей, получения данных и пересчёта скидок. Такой подход позволил разграничить ответственность между модулями: контекст отвечает за связь с базой данных, модели – за представление данных, сервис – за логику обработки, а WPF-приложение – за взаимодействие с пользователем.

Для получения кода требуемого качества применяются соглашения по оформлению и структуре, принятые для языка C# и технологии WPF. Используется единый стиль именования классов, свойств и методов, поддерживается читаемость кода за счёт логичного разбиения на методы и избежания дублирования. При изменении функциональности производится рефакторинг: вынос повторяющихся участков в отдельные методы, упрощение сложных конструкций, уточнение имён, отражающих назначение элементов. Благодаря этому код остаётся понятным и удобным для сопровождения.

Ключевым методом обеспечения качества является автоматизированное модульное тестирование. Для всех основных операций сервиса разработаны unit-тесты в проекте ParshakovaTests. Тестами покрываются сценарии добавления, изменения и удаления партнёров и продаж, а также пересчёта скидок и выборки данных. Использование тестов позволяет поддерживать стабильность функциональности при доработках, своевременно обнаруживать регрессии и гарантировать соответствие реализованного кода исходным требованиям проекта.

3.1.4. Инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования

В рассматриваемом проекте инспектирование компонент программного обеспечения выполняется для библиотечных классов ParshakovaLib, окон и обработчиков событий в приложении ParshakovaWpf, а также тестовых классов ParshakovaTests.

Целью инспекции является проверка соответствия исходного кода принятым соглашениям по стилю и структуре, устранение потенциальных дефектов и повышение общей читаемости решений.

Основным видом инспекции является ручной просмотр кода в интегрированной среде разработки с использованием чек-листов по стилю C# и архитектурным требованиям проекта.

Особое внимание уделяется корректному использованию контекста базы данных, обработке ошибок, разделению ответственности между слоями и аккуратному обращению с зависимостями. Дополнительно используется ревью кода при внесении существенных изменений: новые или переработанные методы сервиса, изменение логики работы главного окна и окон редактирования, введение новых тестов анализируются с точки зрения понятности и соответствия принятой архитектуре.

В качестве актуальных стандартов кодирования используются рекомендации Microsoft по языку C# и платформе .NET. Они определяют формат именования классов, методов и свойств, расположение членов класса, подходы к обработке исключений и ведению логических проверок. На уровне проекта дополнительно соблюдаются требования к разбиению кода на отдельные файлы и пространства имён: контекст и модели выделены в ParshakovaLib.Context и ParshakovaLib.Models, бизнес-логика – в сервисе, код

пользовательского интерфейса – в отдельных классах окон WPF, а тесты – в отдельном проекте.

Для поддержки инспекции применяются встроенные средства среды разработки (Visual Studio или аналогичной IDE), включающие подсветку синтаксиса, проверку базовых предупреждений компилятора и анализатор стиля. При необходимости могут использоваться статические анализаторы, ориентированные на C#, которые помогают выявлять потенциально опасные конструкции, нарушения соглашений по стилю, неиспользуемый код и возможные проблемы с надёжностью. Результаты таких проверок учитываются при последующем ручном ревью кода, что повышает общее качество программного продукта.

3.1.5. Выполнение отладки программного модуля с использованием специализированных программных средств;

В ходе разработки программного обеспечения для учёта партнёров и их продаж отладка выполнялась с использованием специализированных средств интегрированной среды разработки. Основным инструментом является отладчик, встроенный в IDE, позволяющий запускать приложение в пошаговом режиме, анализировать значения переменных и отслеживать последовательность вызовов методов.

Отладка применялась к WPF-приложению ParshakovaWpf, реализующему пользовательский интерфейс.

На подготовительном этапе в коде устанавливались точки останова в ключевых местах: при создании и сохранении объектов партнёров и продаж, при пересчёте скидок, при загрузке данных в элементы интерфейса.

Полный процесс отладки основных модулей и сценариев работы приложения подробно изложен в пункте 3.1.2.4.

3.1.6. Осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения

В данном проекте разработка тестовых наборов и сценариев основана на функциональных требованиях к библиотеке ParshakovaLib и её взаимодействию с базой данных PostgreSQL.

На первом этапе были выделены ключевые операции сервиса: работа с партнёрами (добавление, поиск по имени, обновление, удаление), работа с продажами (добавление, изменение, удаление, выборка по партнёру), а также пересчёт скидок в зависимости от общего объёма продаж. Для каждой операции определены ожидаемые результаты и возможные пограничные ситуации.

На основе этих требований сформирован набор unit-тестов в проекте ParshakovaTests. Тесты проверяют, что при добавлении партнёра запись действительно появляется в базе данных, при обновлении изменённые данные сохраняются, при удалении партнёра совместно удаляются связанные с ним продажи.

Для операций с продажами контролируется, что новые записи корректно создаются, изменённые значения сохраняются, а удалённые продажи больше не возвращаются при выборке.

Отдельный тестовый сценарий проверяет корректность расчёта скидок при различных суммарных объёмах продаж: для нескольких партнёров создаются разные наборы продаж, после пересчёта скидок результаты сравниваются с ожидаемыми значениями.

Ниже можно ознакомится со списком тестов и их описанием.

Таблица 1 – Unit тесты

Название теста	Метод который тестируется	Краткое описание теста
UpdateDiscounts_ SetsCorrectDiscount_	UpdateDiscounts()	Проверяет расчет скидок

ForDifferentTotalSales		для партнеров с разным объемом продаж (1000, 20000, 100000, 400000). Ожидаемые скидки: 0%, 5%, 10%, 15%
GetPartnerByProperties_ ReturnsPartnerByName	GetPartnerByProperties()	Создает тестового партнера и проверяет, что метод возвращает правильного партнера при поиске по имени
AddPartner_ AddsNewPartner	AddPartner()	Добавляет нового партнера в БД и проверяет, что он успешно сохранился
UpdatePartner_ UpdatesExistingPartner	UpdatePartner()	Обновляет имя существующего

		партнера и проверяет, что изменения сохранились
DeletePartner_ DeletesPartnerAndSales	DeletePartner()	Удаляет партнера и связанные с ним продажи, проверяет что они удалились из БД
AddSale_ AddsNewSale	AddSale()	Добавляет новую продажу для существующего партнера и проверяет ее сохранение
UpdateSale_ UpdatesExistingSale	UpdateSale()	Обновляет количество товара в существующей продаже и проверяет изменения
DeleteSale_ DeletesExistingSale	DeleteSale()	Удаляет существующую

		продажу и проверяет что она удалилась из БД
GetSales_ ReturnsSalesForPartner	GetSales()	Создает двух партнеров с продажами и проверяет, что для первого партнера возвращаются только его продажи (2 из 3)
GetPartners_ ReturnsPartnersWithDiscounts	GetPartners()	Проверяет, что метод возвращает партнеров вместе с их скидками
LoadPartners_ ReturnsPartnersWithDiscounts	LoadPartners()	Проверяет, что метод загружает партнеров вместе с их скидками
SaveChanges_	SaveChanges()	Проверяет, что

DoesNotThrow_ WhenContextIsNull		метод не выбрасывает исключение при передаче null контекста

Процесс проведения тестирования включает подготовку тестовых данных, запуск тестовых методов и анализ итоговых отчётов. Для минимизации влияния тестов на рабочие данные используется вспомогательный метод очистки, удаляющий все записи, созданные в ходе тестирования. Это позволяет повторно запускать полный набор тестов без накопления служебных записей. Тесты выполняются в рамках средства модульного тестирования, входящего в состав среды разработки, а их результаты отображаются в виде списка успешно пройденных и проваленных проверок.

В результате разработки и регулярного выполнения test-наборов обеспечивается постоянный контроль качества реализованных функций. При внесении изменений в код библиотеки или в структуру базы данных тесты позволяют оперативно выявлять нарушения ожидаемого поведения. Такой подход снижает риск появления скрытых ошибок и повышает надёжность программного обеспечения при его дальнейшей эксплуатации и развитии.

Полную информацию о тестах проекта можно увидеть в пункте 3.1.2.4.

3.1.7. Интегрирование программных модулей

Интеграция модулей в рассматриваемом проекте реализована поэтапно и опирается на заранее сформированную архитектуру. В качестве основы используется разделение на три основных слоя: база данных PostgreSQL, библиотека ParshakovaLib и клиентское приложение ParshakovaWpf. На первом этапе была выполнена интеграция библиотеки с базой данных: класс контекста ApplicationContext настроен на использование конкретной базы, а модели

партнёров, продаж и скидок сопоставлены с соответствующими таблицами. На этом уровне проводилась проверка корректности строк подключения, структуры сущностей и настроек навигационных свойств.

На следующем этапе были интегрированы сервисные методы библиотеки с пользовательским интерфейсом WPF-приложения. Главное окно и окна добавления и редактирования данных получают доступ к сервису, который, в свою очередь, взаимодействует с контекстом базы данных. При выборе партнёра в списке выполняется запрос к библиотеке для загрузки его продаж, при нажатии кнопок добавления, изменения и удаления вызываются методы сервиса, отвечающие за соответствующие операции. Таким образом, логика работы интерфейса построена поверх уже реализованных и оттестированных функций библиотеки.

В процессе интеграции выполнялось интеграционное тестирование: проверялась совместная работа модулей в реальных сценариях использования. Проверялось, что добавление данных через окна WPF корректно отражается в базе данных, информация обновляется в интерфейсе, а пересчёт скидок производится автоматически при изменении продаж. Особое внимание уделялось обработке ошибок при недоступности базы данных или некорректных данных на уровне интерфейса. По итогам интеграции был получен единый программный комплекс, в котором все модули взаимодействуют согласованно и обеспечивают выполнение поставленных задач.

3.1.8. Верификация и аттестация программного обеспечения

Верификация разработанного программного обеспечения выполняется путём сопоставления реализаций с исходными требованиями к системе учёта партнёров и продаж.

На этапе верификации анализируются структура базы данных, возможности интерфейса и логика работы сервиса. Проверяется полнота реализации функций: наличие операций добавления, редактирования и удаления партнёров и продаж, корректный расчёт скидок на основе объёма реализации, отображение актуальной информации в главном окне. Кроме того, оценивается соответствие кода архитектурным решениям и соглашениям по стилю.

Важной частью верификации являются результаты модульного и интеграционного тестирования. Набор unit-тестов демонстрирует корректность выполнения операций библиотеки ParshakovaLib, а проверки, проводимые в ходе интеграции с WPF-приложением, подтверждают правильность совместной работы модулей. При обнаружении несоответствий в ходе тестов или визуальной проверки поведения интерфейса вносятся корректировки в код и повторно выполняются тестовые сценарии. Это позволяет убедиться, что ошибки устранены и функциональность реализована в полном объёме.

Аттестация программного обеспечения связана с оценкой готовности системы к практическому использованию в учебных и производственных целях. Для этого проводится демонстрация работы приложения: выполняются типичные операции по ведению списка партнёров, регистрации продаж и контролю скидок. Оцениваются стабильность работы программы, удобство интерфейса, скорость реакции на действия пользователя и предсказуемость результатов. По итогам проверки формируется вывод о том, что программный продукт соответствует заявленным требованиям и может быть использован в качестве готового решения учебного проекта.

ЗАКЛЮЧЕНИЕ

В результате прохождения производственной практики с 09 марта по 16 марта 2026 года на базе Индивидуального предпринимателя Паршаковой Екатерины Николаевны (ИП Паршакова Е.Н.) были закреплены теоретические знания и расширены профессиональные умения в области интеграции программных модулей.

По итогам практики получены следующие результаты:

— Разработан программный модуль для автоматизации учёта партнёров и их продаж, включающий базу данных PostgreSQL, библиотеку классов на языке C# с бизнес-логикой и WPF-приложение с пользовательским интерфейсом.

— Освоен полный цикл разработки программного обеспечения: от анализа требований и проектирования до реализации, интеграции, отладки и тестирования.

— Получен практический опыт применения системы контроля версий Git для управления исходным кодом проекта.

— Закреплены навыки работы с системами управления базами данных (PostgreSQL), средой разработки (Visual Studio 2022), технологиями .NET, Entity Framework Core, WPF, а также инструментами модульного тестирования (MSTest).

— Выполнена интеграция разработанных модулей в единый программный комплекс, проведена отладка с использованием специализированных средств, разработаны тестовые наборы и сценарии, осуществлено инспектирование компонент на предмет соответствия стандартам кодирования.

— Проведённая верификация и аттестация программного обеспечения подтвердили его соответствие исходным требованиям и готовность к практическому использованию.

Таким образом, задачи, поставленные в индивидуальном задании, выполнены в полном объеме, цель практики — комплексное освоение видов профессиональной деятельности по интеграции программных модулей и приобретение практического опыта в области разработки, отладки и тестирования программного обеспечения — достигнута. Полученные знания и навыки будут использованы в дальнейшем обучении и профессиональной деятельности.

СПИСОК ЛИТЕРАТУРЫ

Информационное обеспечение обучения.

1. ГОСТ Р ИСО/МЭК 9294-93. Информационная технология. Руководство по управлению документированием программного обеспечения.

2. ГОСТ Р ИСО/МЭК 9126-93. Информационная технология. Оценка программной продукции. характеристика качества и руководство по их применению.

3. ГОСТ Р ИСО/МЭК 9127-94. Системы обработки информации. Документация пользователя и информация на упаковке для потребительских программных пакетов.

4. ГОСТ Р ИСО/МЭК 8631-94. Информационная технология. Программные конструктивы и условные обозначения для их представления.

5. ГОСТ Р ИСО/МЭК 12119:2000. Информационная технология. Пакеты программных средств. Требования к качеству и испытания.

6. ГОСТ 2.108 - 68 - Спецификация (ЕСКД)

7. ГОСТ 2.105 - 95 - Общие требование к текстовым документам. (ЕСКД)

8. ГОСТ 2.103 - 68 - Стадии разработки (ЕСКД)

9. ГОСТ 2.106 - 68 - Ведомость спецификаций (ЕСКД)

10. ГОСТ - 7.1- 84 - Библиографическое описание документа. Общие требования и правила составления. (ЕСКД)

11. ГОСТ 2.004 - 88 - Общие требования к выполнению конструкторских и технологических документов на печатающих и графических устройствах вывода ЭВМ. (ЕСКД)

12. ГОСТ 19.101-77 - Виды программ и программных документов

13. ГОСТ 19.103-77 - Обозначение программ и программных документов.

14. ГОСТ 19.102 - 77 - Стадии разработки

15. ГОСТ 19.104 - 78 - Основные надписи

16. ГОСТ 19.105 - 78 - Общие требования к программным документам

17. ГОСТ 19.201 - 78 - Техническое задание.

18. ГОСТ 19.202 - 78 - Спецификация

19. ГОСТ 19.781- 90 - Термины и определения

20. ГОСТ 19.701-90 - Схемы алгоритмов, программ данных и схем. Условные обозначения и правила выполнения

21. ГОСТ Р ИСО/МЭК 12207-2010 «Информационная технология. Системная и программная инженерия. Процессы жизненного цикла программных средств»

22. Федорова Г. Н. Участие в интеграции программных модулей. Учебное пособие / Федорова Г.Н. – М.Ж Издательский дом «Академия», 2016 – 304с.

23. Федорова Г. Н. Разработка, внедрение и адаптация программного обеспечения отраслевой направленности: Учебное пособие. / Федорова Г.Н. - М.:КУРС, НИЦ ИНФРАМ, 2016. - 336 с. http://znanium.com/catalog.php?bookinfo=544732

24. Оценка качества программного обеспечения: Практикум: Учебное пособие / Б.В. Черников, Б.Е. Поклонов; Под ред. Б.В. Черникова - М.: ИД ФОРУМ: НИЦ Инфра-М, 2012. - 400 с. http://znanium.com/bookread2.php?book=315269

25. Гуриков С. Р. Введение в программирование на языке Visual C#: Учебное пособие / С.Р. Гуриков. - М.: Форум: НИЦ ИНФРА-М, 2013. - 448 с. http://znanium.com/bookread2.php?book=232661

26. Исаченко О.В. Программное обеспечение компьютерных сетей : учебное пособие. М. : ИНФРА-М, 2013. - 117 с. - (СПО).

27. Стандартизация, сертификация и управление качеством программного обеспечения: Учебное пособие / Ананьева Т.Н., Новикова Н.Г., Исаев Г.Н. - М.:НИЦ ИНФРА-М, 2016. - 232с http://znanium.com/bookread2.php?book=541003

28. Гагарина Л.Г. Технология разработки программного обеспечения: Учебное пособие / Л.Г. Гагарина, Б.Д. Виснадул, Е.В. Кокорева. М.: ООО

"Научно-издательский центр ИНФРА-М", 2013. 400 с. URL: http :// znanium. com / catalog. php ? bookinfo =389963 10

29. Немцова Т. И. Программирование на языке высокого уровня. Программир. на языке С++[Электронный ресурс]: Уч. пос. / Т.И.Немцова и др.; Под ред. Л.Г.Гагариной. М.: ИД ФОРУМ: ИНФРА-М, 2012. 512 с. URL: http://znanium.com/catalog.php?bookinfo=244875;

30. Рудаков А.В. Технология разработки программных продуктов: Учеб.пособие для студ. сред. проф. образования. М.: Издательство «Академия», 2010. 208 с.

31. Черников Б.В. Управление качеством программного обеспечения [Электронный ресурс]: Учебник / Б.В. Черников. - М.: ИД ФОРУМ: ИНФРА-М, 2012. - 240 с.. URL: http://znanium.com/catalog.php?bookinfo=256901.

32. Эйдлина Г.М. Delpfi: программирование в примерах и задачах[Электронный ресурс]: Практикум / Г.М. Эйдлина, К.А. Милорадов. М.: ИЦ РИОР: НИЦ Инфра-М, 2012. 116 с. URL: http :// znanium. com / catalog. php ? bookinfo =319046

Дополнительные источники:

1. Гагарина Л.Г. Разработка и эксплуатация автоматизированных информационных систем [Электронный ресурс]: Учебное пособие / Л.Г. Гагарина. М.: ИД ФОРУМ: НИЦ Инфра-М, 2013.384с. URL: http://znanium.com/catalog.php?bookinfo=368454;

2. Голицына О. Л. Программное обеспечение[Электронный ресурс]: Учебное пособие / О.Л. Голицына, Т.Л. Партыка, И.И. Попов. - 3-е изд., перераб.и доп. М.: Форум, 2010.448с. URL: http://znanium.com/catalog.php?bookinfo=201030;

3. Роббинс, Д. Отладка Windows-приложений [Электронный ресурс] / Д. Роббинс, пер. с англ. М.: ДМК Пресс, 2009. 448 с., ил. URL: http://znanium.com/catalog.php?bookinfo =407747

Интернет-ресурсы:

1. Технология программирования: [Электронныйресурс]. URL: http :// glossary. ru / cgi - bin / gl _ sch 2. cgi ? RSl) turujoo ! vwujwgssowuigto 9

2. Технология программирования:[Электронный ресурс].URL: http ://2 programmer. ru/l1?start=3

3. Информатика:[Электронный ресурс]. URL: http:// klgtu. ru / students / literature / inf _ asu /600. html

4. Языки программирования: [Электронный ресурс]. URL: http://maksakov-sa.ru/TehProgram/index.html

5.Информатика: [Электронный ресурс]. URL: http://object.newmail.ru/obj1.html

6.Delphi и базы данных: [Электронный ресурс]. URL: http://snkey.net/books/delphi/ch4- 2.html

7. Базы данных: [Электронный ресурс]. URL: http://www.sdteam.com/5/